
Lecture 6: Linear Regression

Md. Shahriar Hussain
ECE Department, NSU



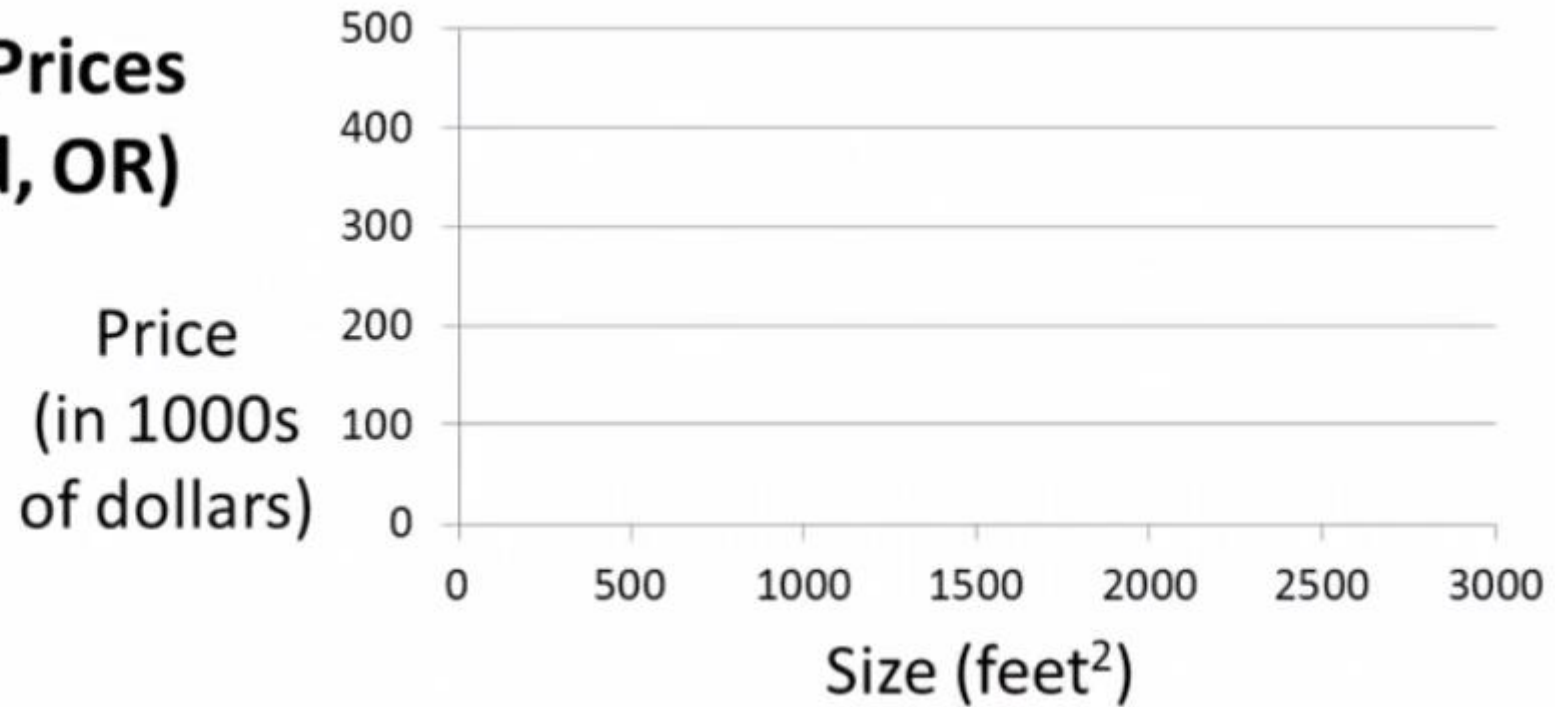
What is Linear Regression

- **Linear regression** is defined as an algorithm that provides a linear relationship between an independent variable and a dependent variable to predict the outcome of future events



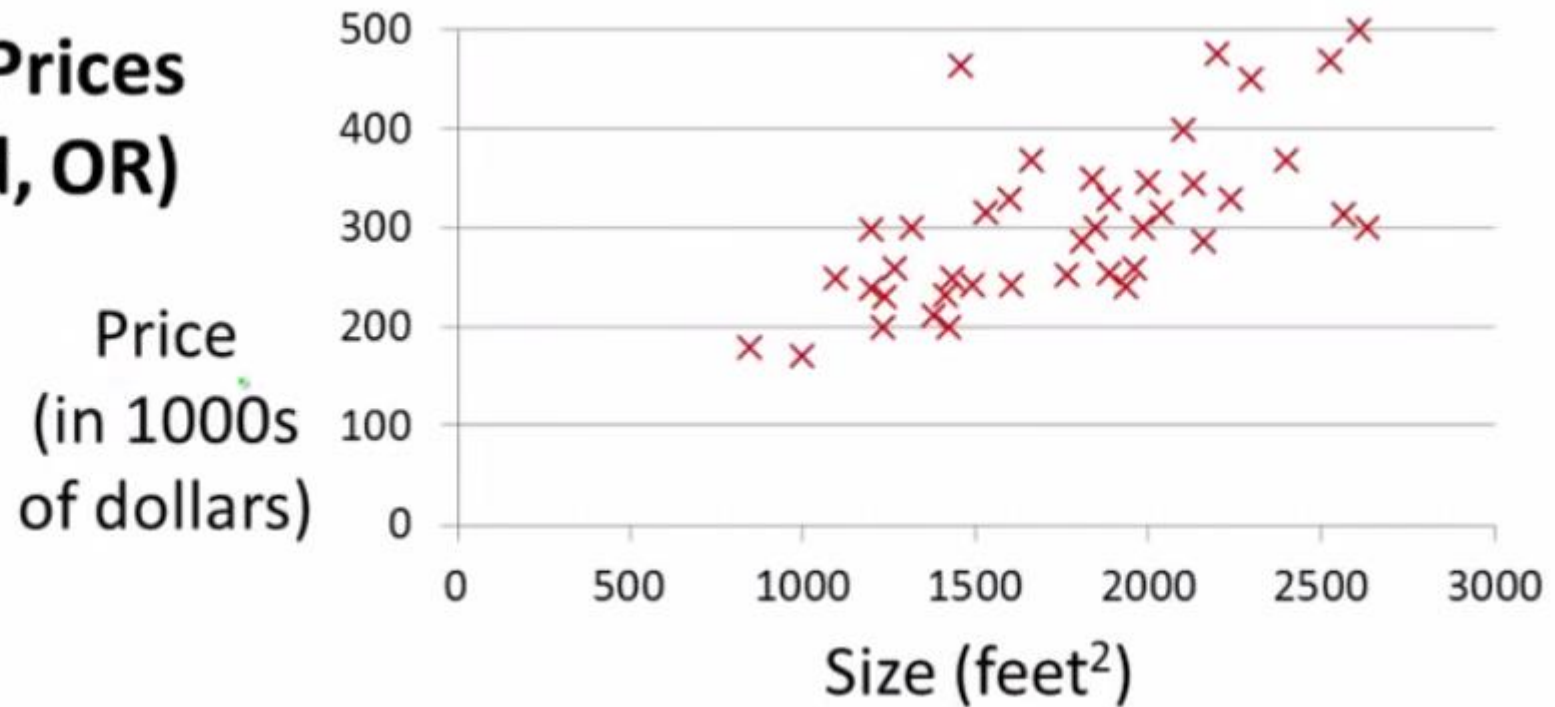
Linear Regression Example

Housing Prices (Portland, OR)



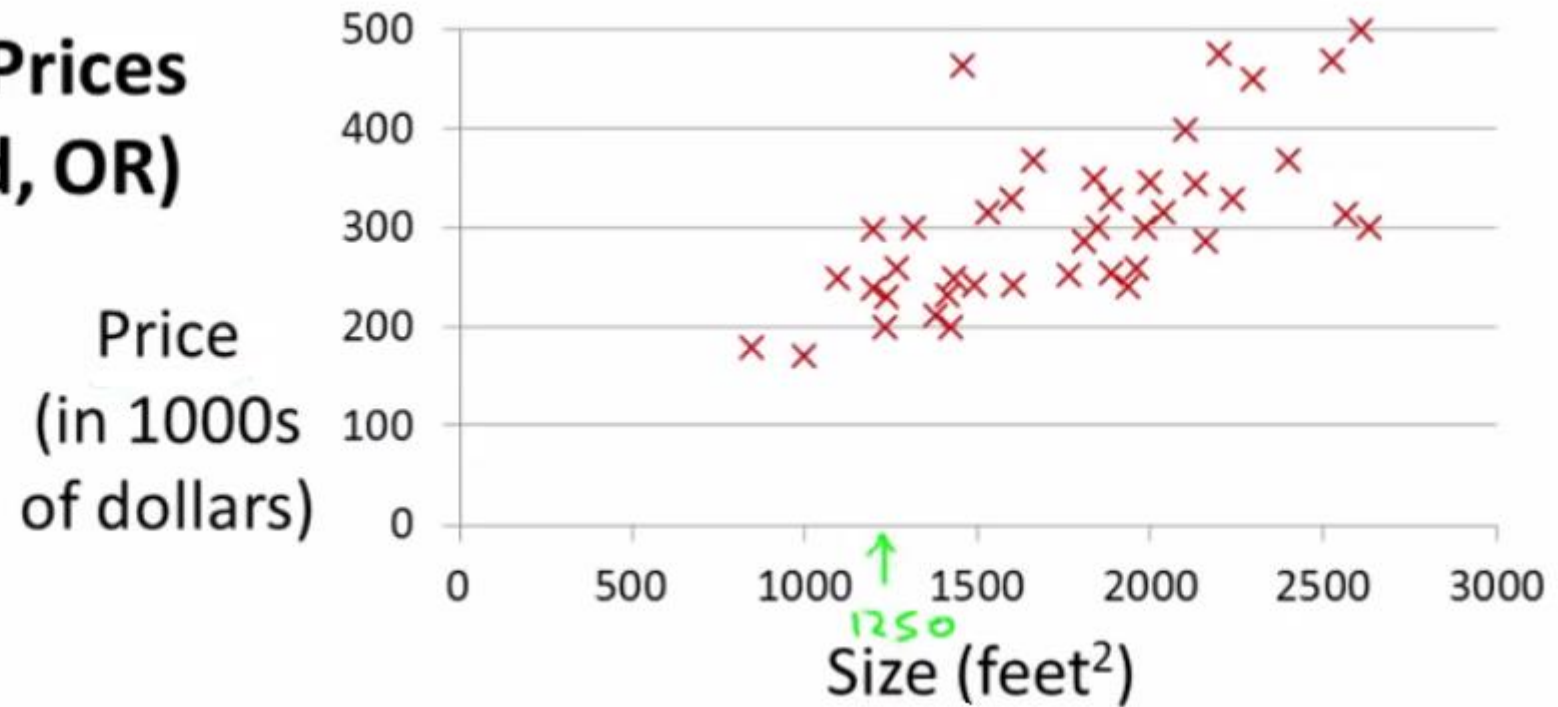
Linear Regression Example

Housing Prices (Portland, OR)



Linear Regression Example

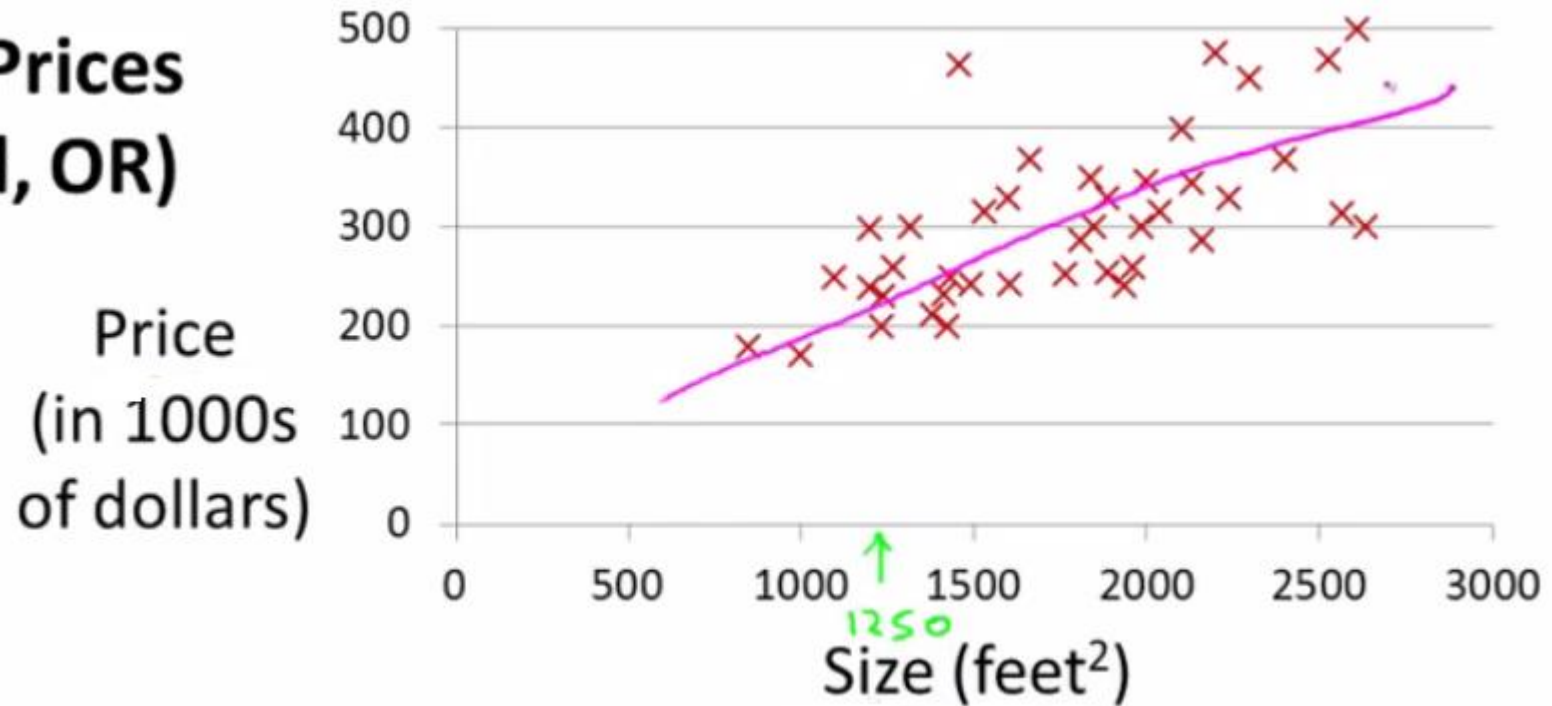
Housing Prices (Portland, OR)



Linear Regression Example

Housing Prices (Portland, OR)

A **Line of best fit/Regression Line** is a straight line that represents the best approximation of a scatter plot of data points



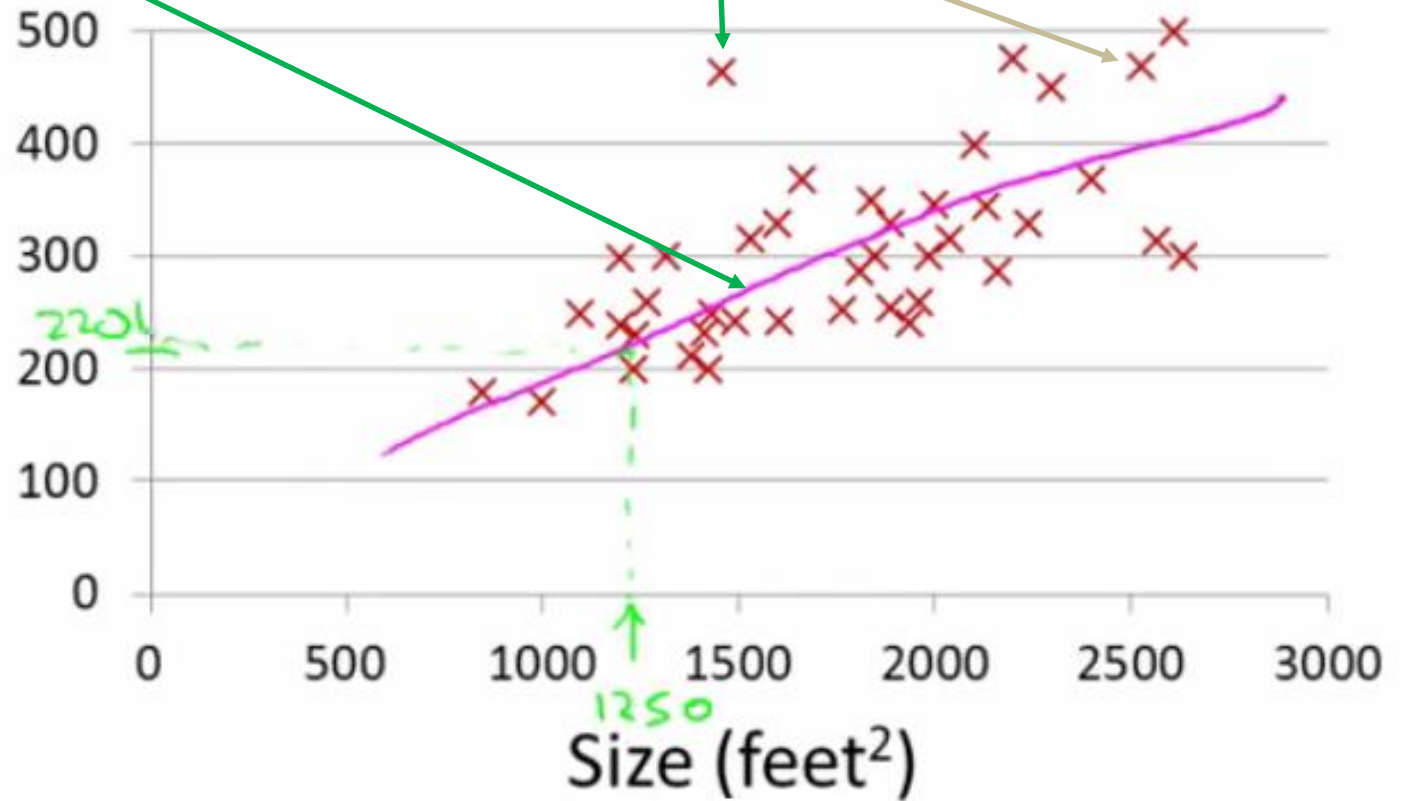
Linear Regression Example

Estimated/Predicted value ($\tilde{y}/\bar{y}$)

Actual/True value (y)/Ground Truth

Housing Prices (Portland, OR)

Price
(in 1000s
of dollars)



Data Set Description

**Training set of
housing prices
(Portland, OR)**

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...

Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable



Data Set Description

Training set of housing prices (Portland, OR)	Size in feet ² (x)	Price (\$) in 1000's (y)
	2104	460
	1416	232
	1534	315
	852	178
	...	...

Notation:

m = Number of training examples

x's = "input" variable / features

y's = "output" variable / "target" variable

(x, y) = One Training Example

(x⁽ⁱ⁾, y⁽ⁱ⁾) = ith Training example

x⁽¹⁾ = 2104

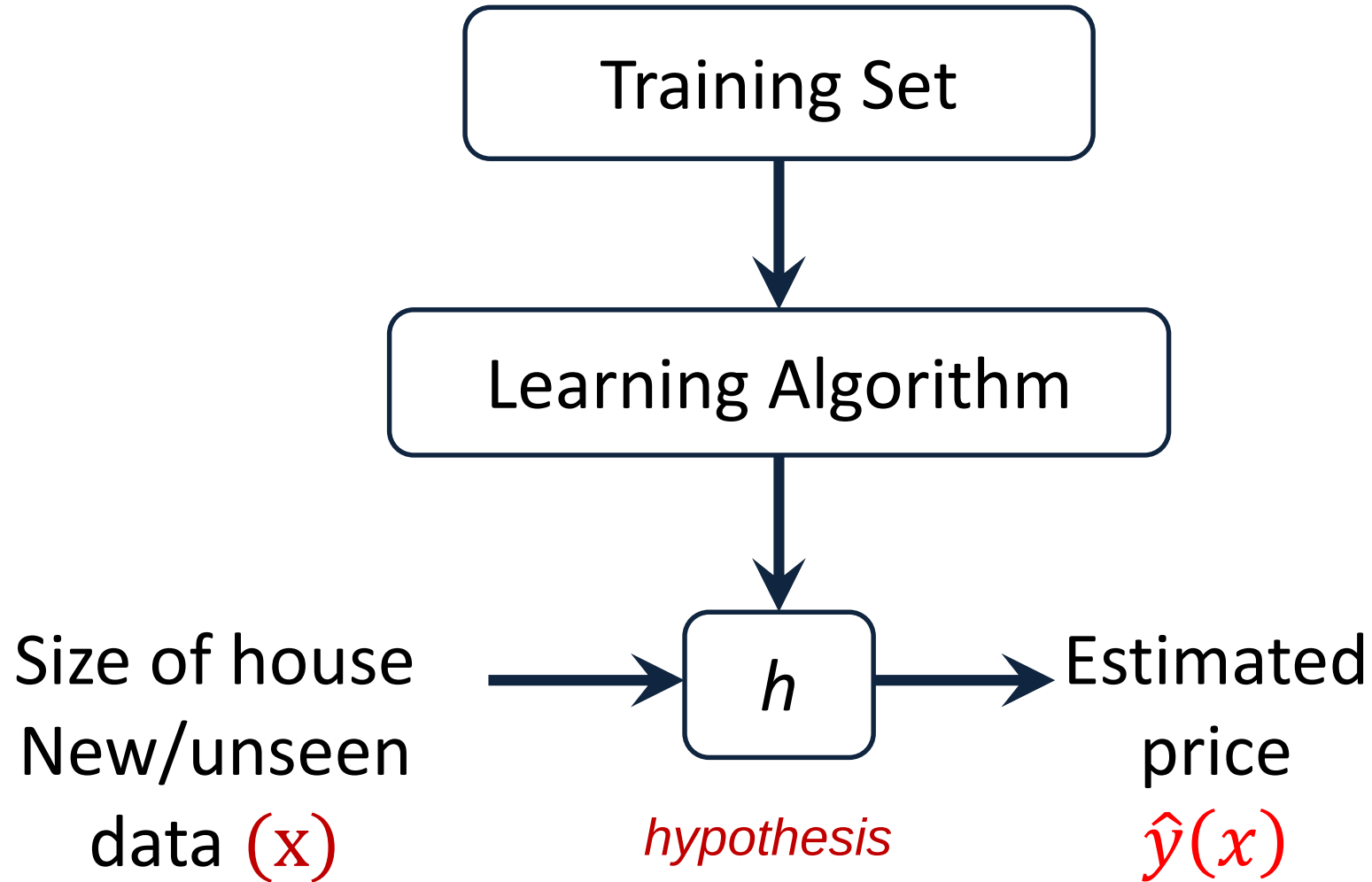
x⁽²⁾ = 1416

y⁽¹⁾ = 460

y⁽²⁾ = 232



Hypothesis



Hypothesis

- How do we represent h ?

$$\hat{y}(x) = h_{\theta}(x) = \theta_0 + \theta_1 x$$

θ_0 and θ_1 : parameters/weights that will be trained/determined by the ML model

Not hyperparameters

θ_0 = intercept/bias/constant

θ_1 = slope/coefficient/gradient

Linear regression with one variable.
Univariate linear regression.



Hypothesis

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

The goal is to choose θ_0 and θ_1 properly so that $h_{\theta}(x)$ is close to y .

- A cost function lets us figure out how to **fit the best straight line** to our data



Hypothesis

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
	...

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

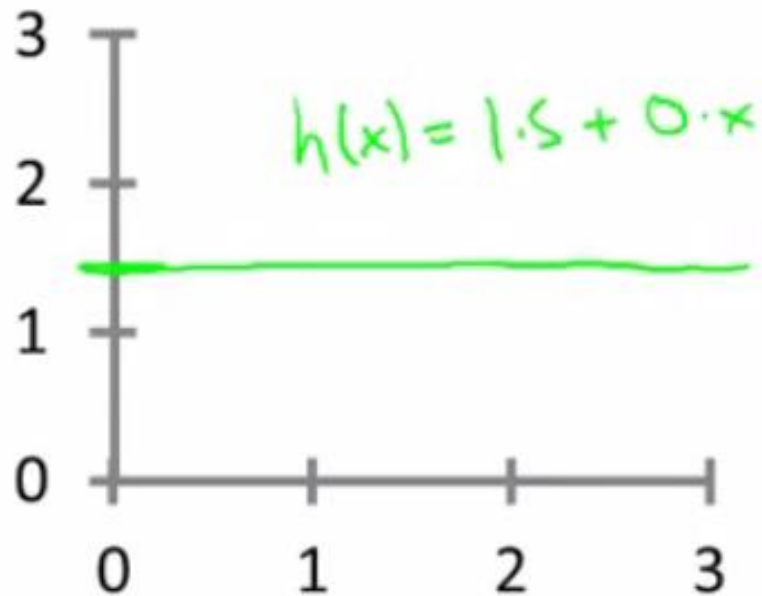
θ_i 's: Parameters

How to choose θ_i 's ?

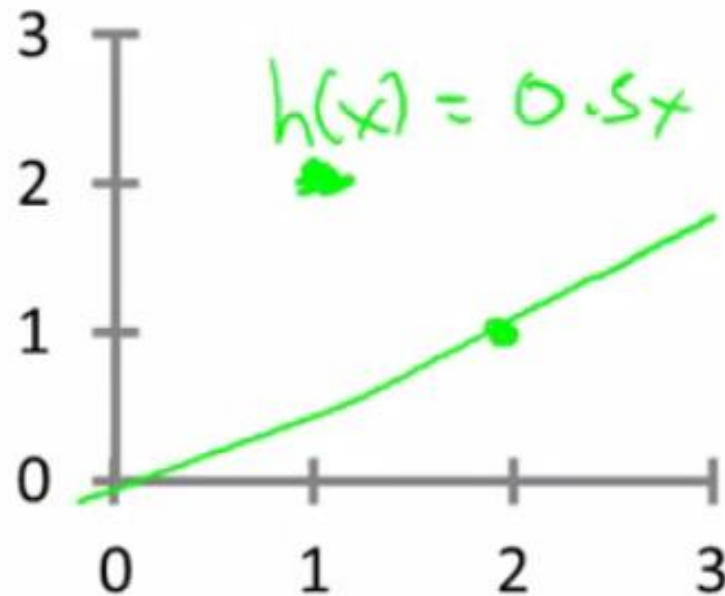


Example

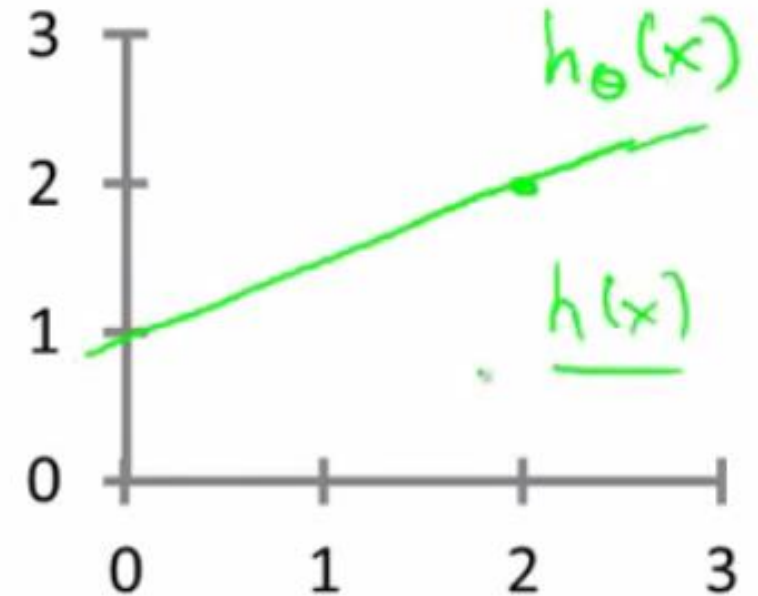
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



$$\theta_0 = 1.5$$
$$\theta_1 = 0$$



$$\theta_0 = 0$$
$$\theta_1 = 0.5$$



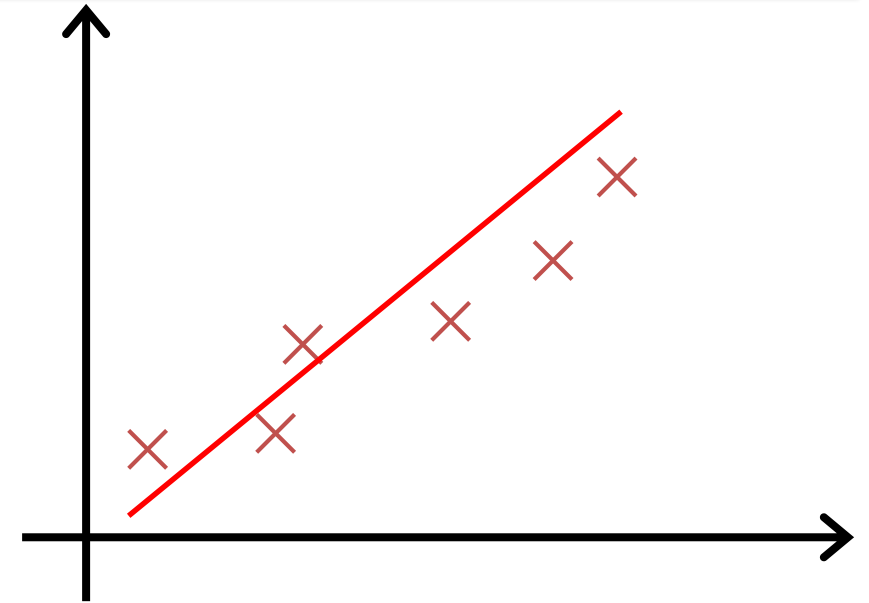
$$\theta_0 = 1$$
$$\theta_1 = 0.5$$



Cost Function

minimize
 $\theta_0 \theta_1$

$$\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$



- We need to choose θ_0 and θ_1 in a way that the result of the function will be minimized for all m training example. This equation is called cost function

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

minimize
 $\theta_0 \theta_1$

$$J(\theta_0, \theta_1)$$



Cost Function

Cost Function:
$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Goal:
$$\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$$

- Here the cost function is called Squared Error cost function
- Minimize squared different between predicted house price and actual house price
- $1/m$ - means we determine the average
- $1/2m$ the 2 makes the math a bit easier, and doesn't change the constants we determine at all (i.e., half the smallest value is still the smallest value!)



Cost Function Calculation

- For simplifications, assumes $\theta_0 = 0$

$$h_{\theta}(x) = \theta_1 x$$

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

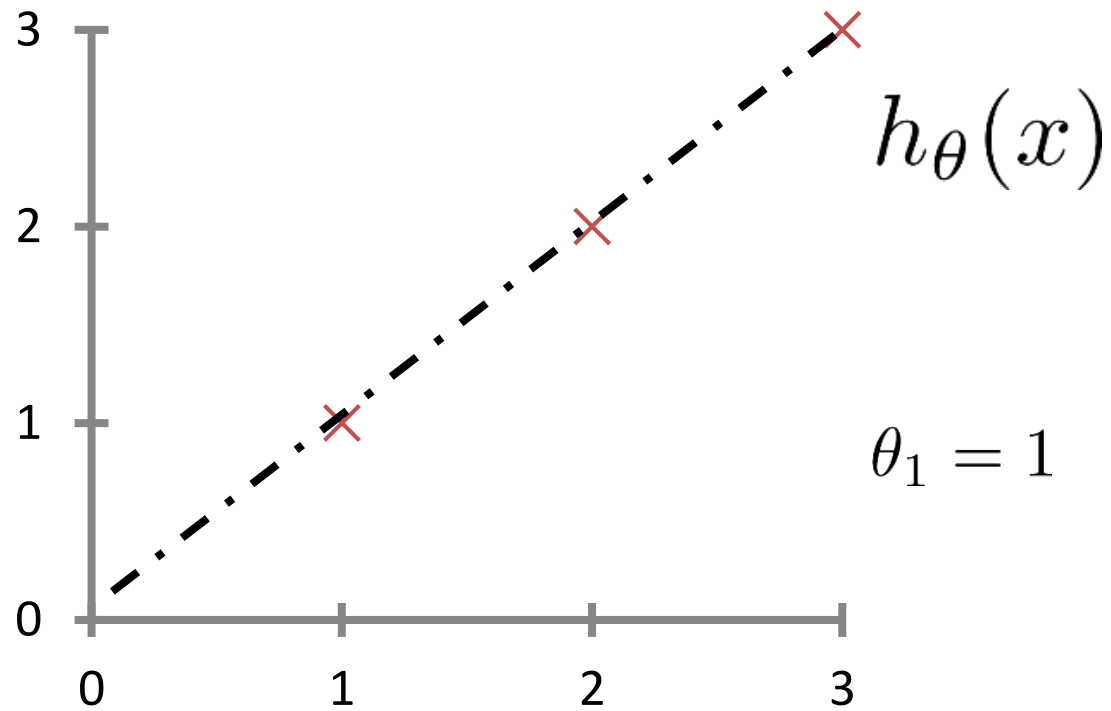
$$\underset{\theta_1}{\text{minimize}} J(\theta_1)$$

Find best values of θ_1 so that $J(\theta_1)$ is minimum



Cost Function Calculation

$$h_{\theta}(x) = \theta_1 x$$

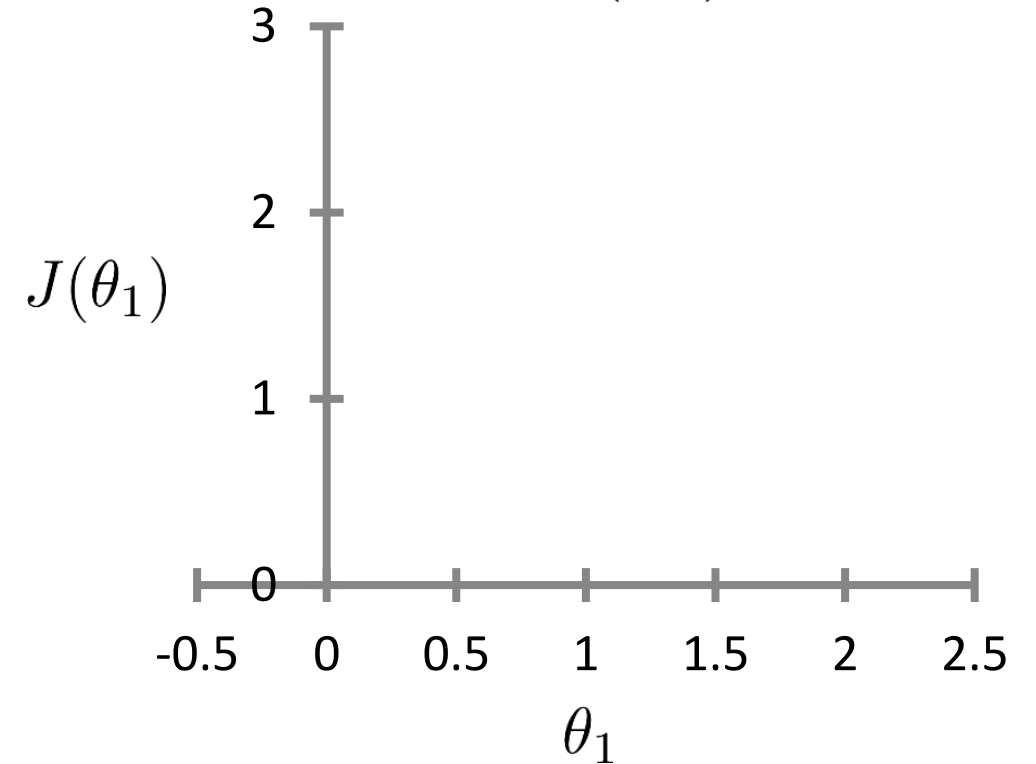


$$\theta_1 = 1$$

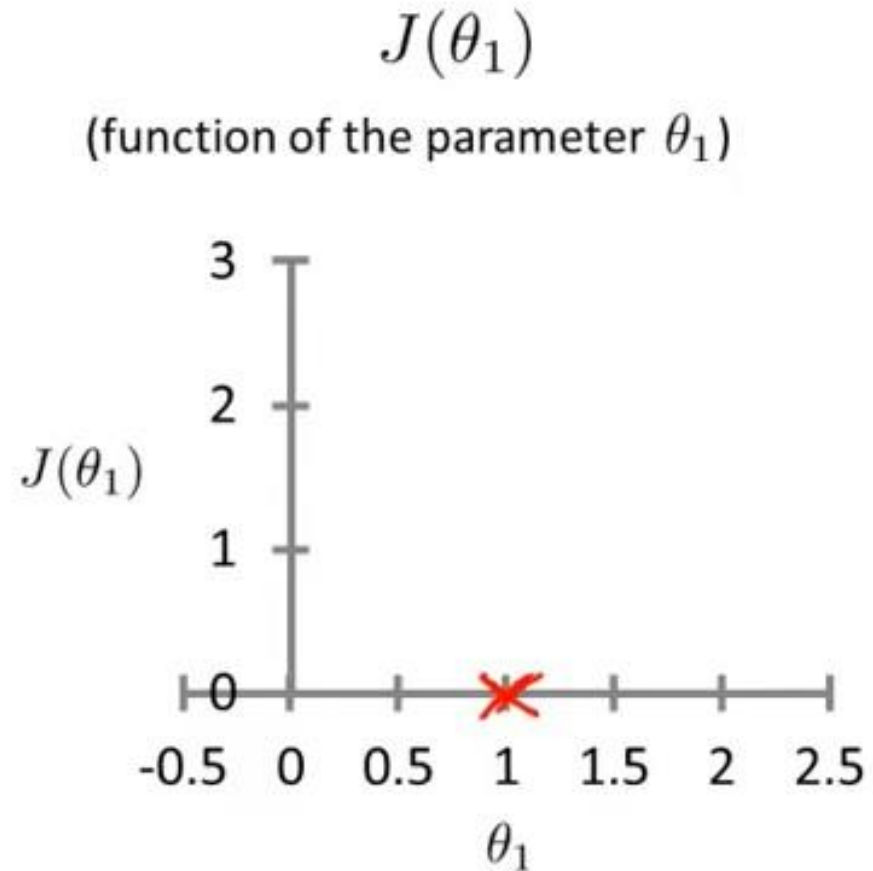
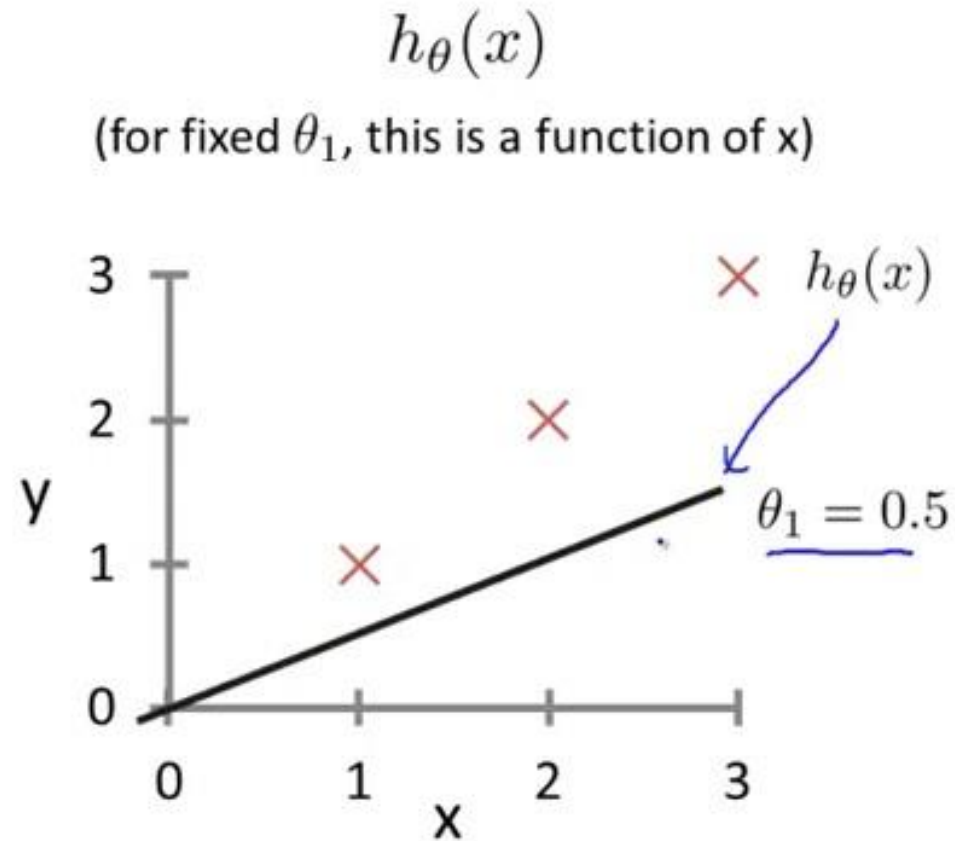
For, $\theta_1 = 1$

$$J(\theta_1) = \frac{1}{2} \cdot 3 [0+0+0] = 0$$

$$J(\theta_1)$$

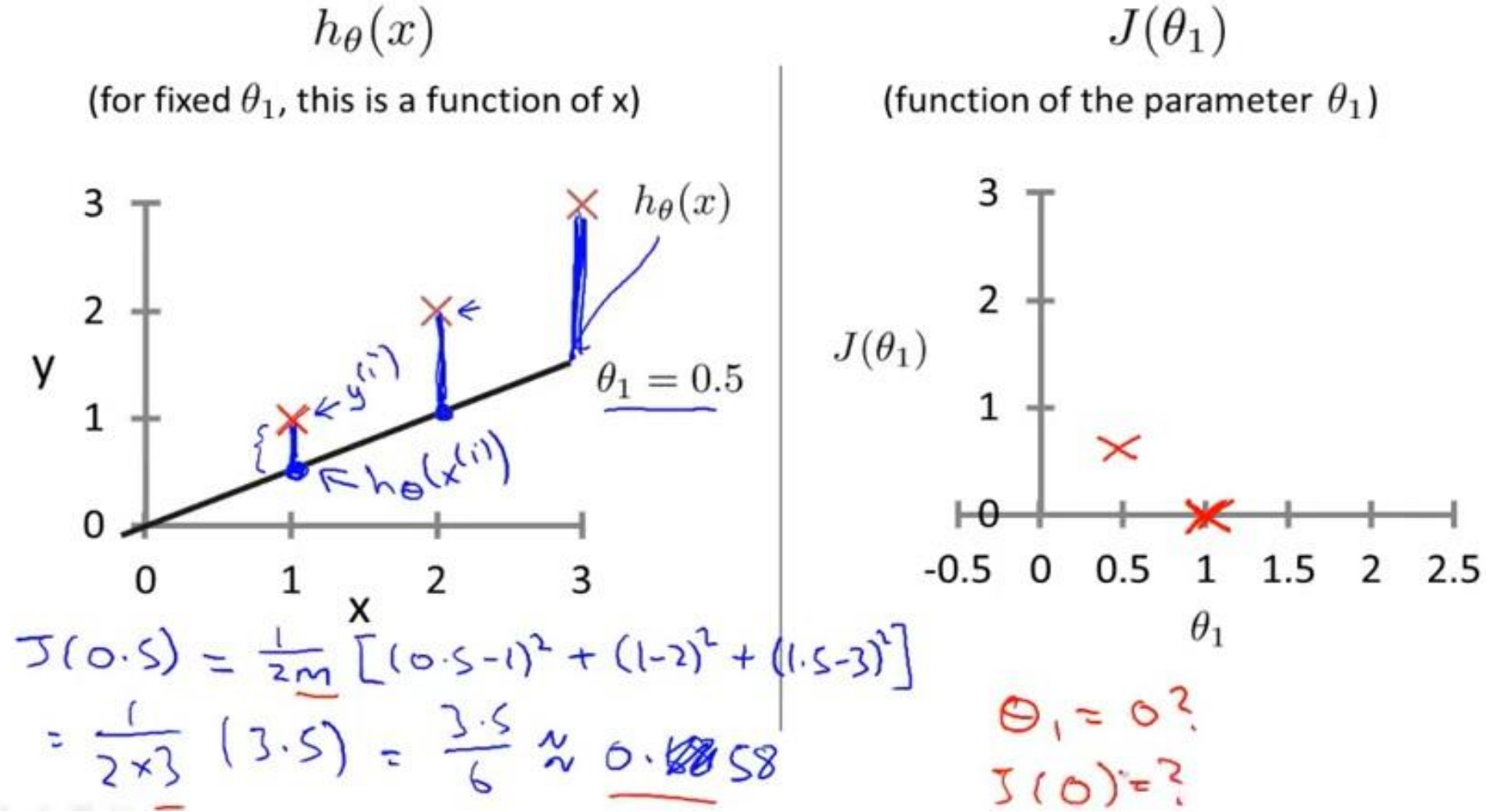


Cost Function Calculation

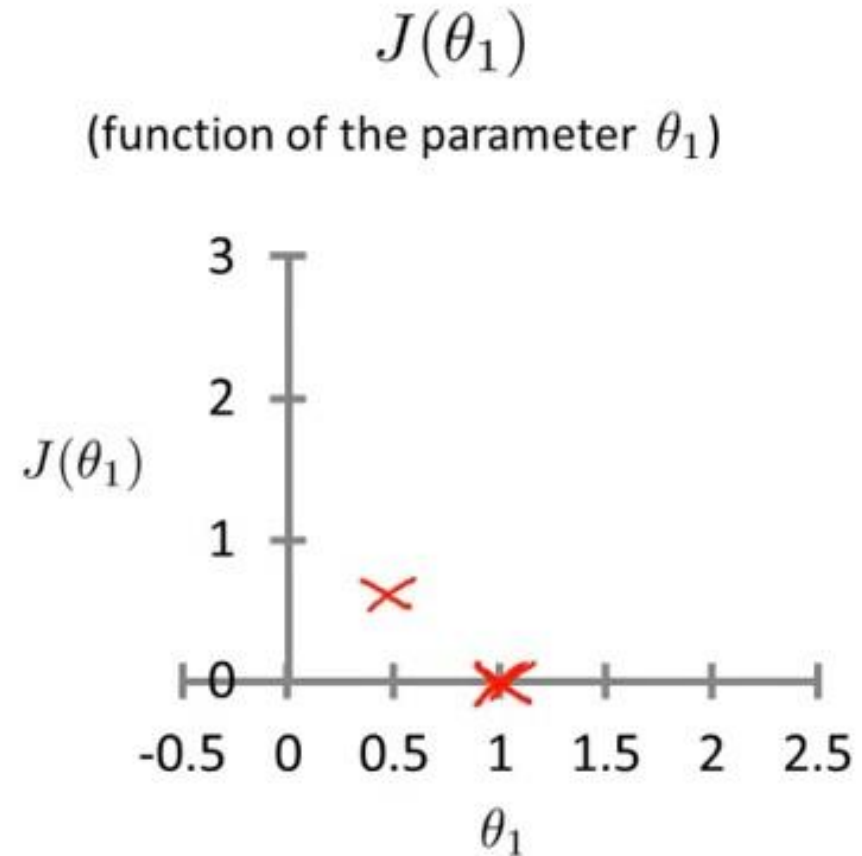
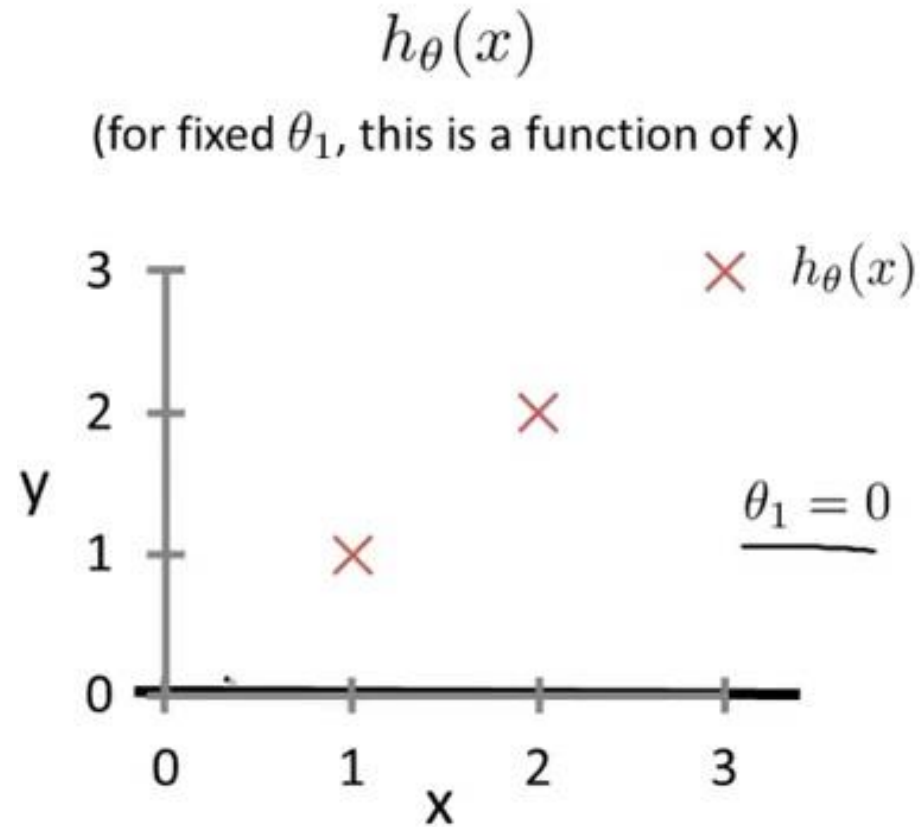


For, $\theta_1 = 0.5$
 $J(\theta_1) = ?$

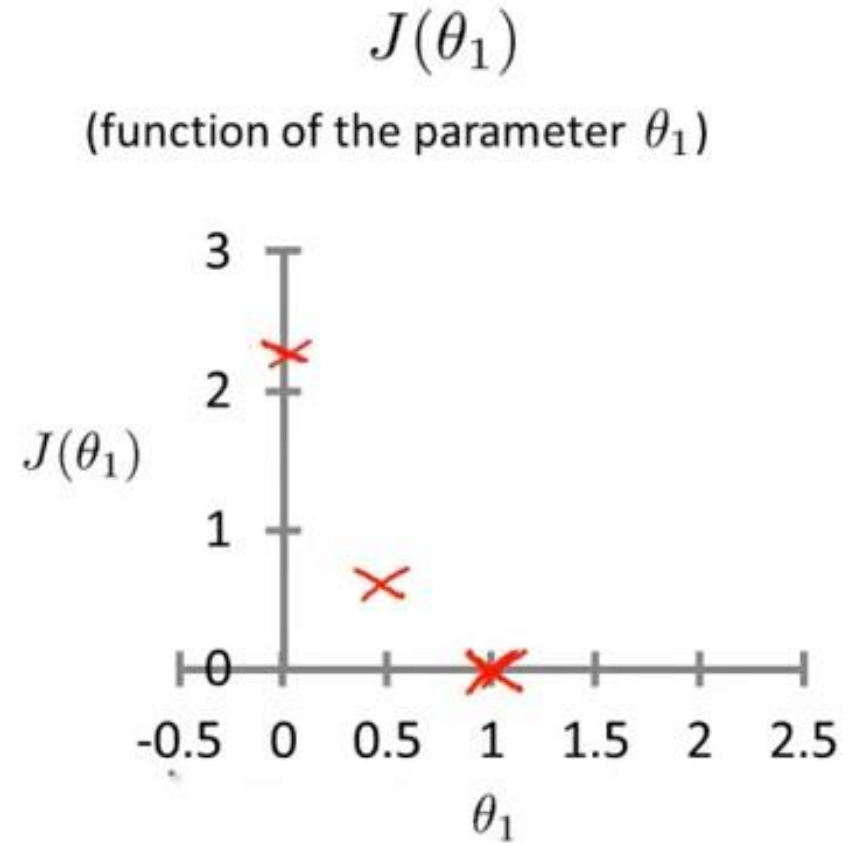
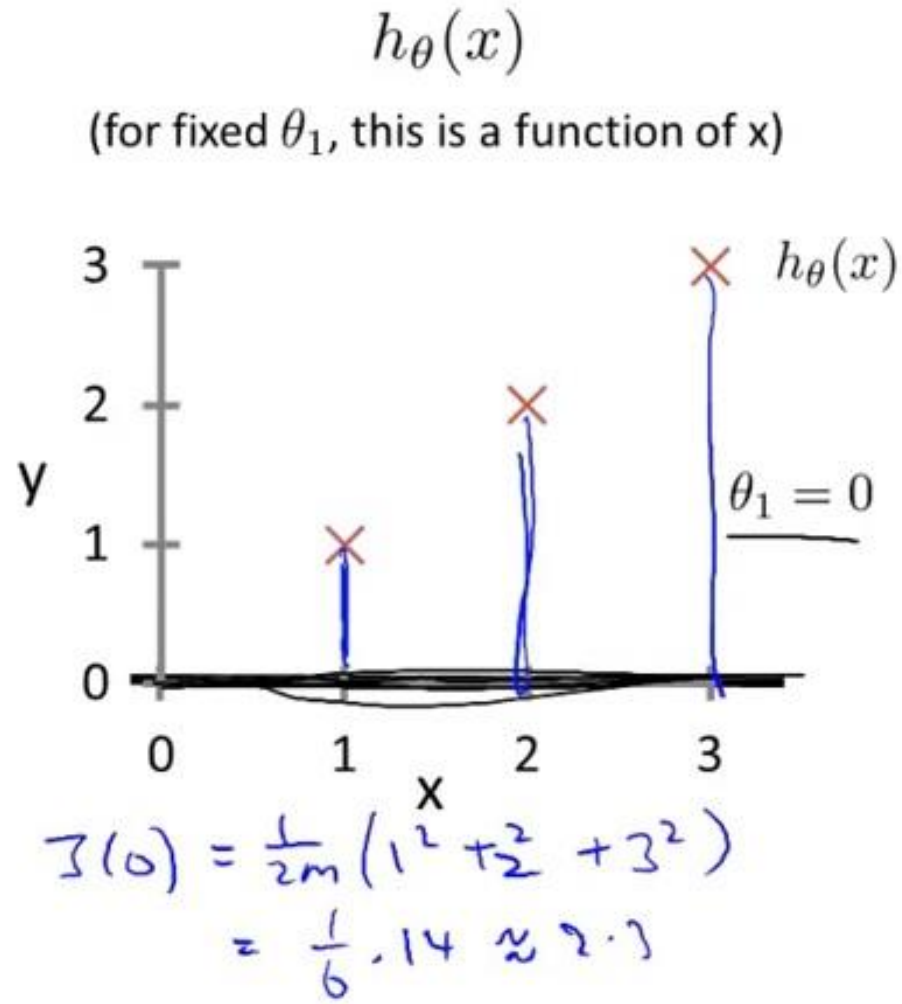
Cost Function Calculation



Cost Function Calculation

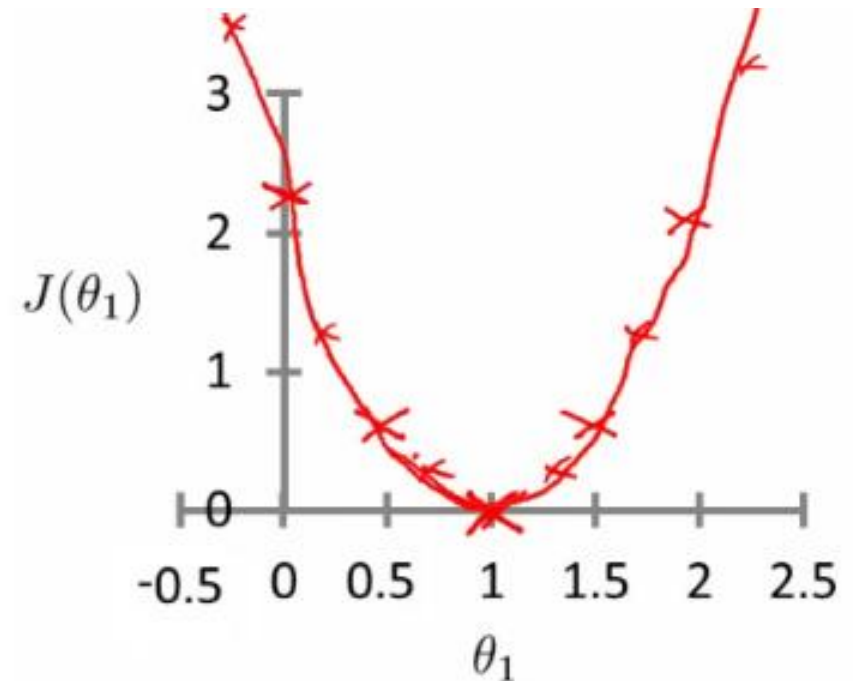


Cost Function Calculation



Cost Function Calculation

- If we compute a range of values plot
 - $J(\theta_1)$ vs θ_1 we get a polynomial (looks like a quadratic)
- The optimization objective for the learning algorithm is find the value of θ_1 which minimizes $J(\theta_1)$
 - So, here $\theta_1 = 1$ is the best value for θ_1
 - **The line which has the least sum of squares of errors is the best fit line**



Important Equations

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

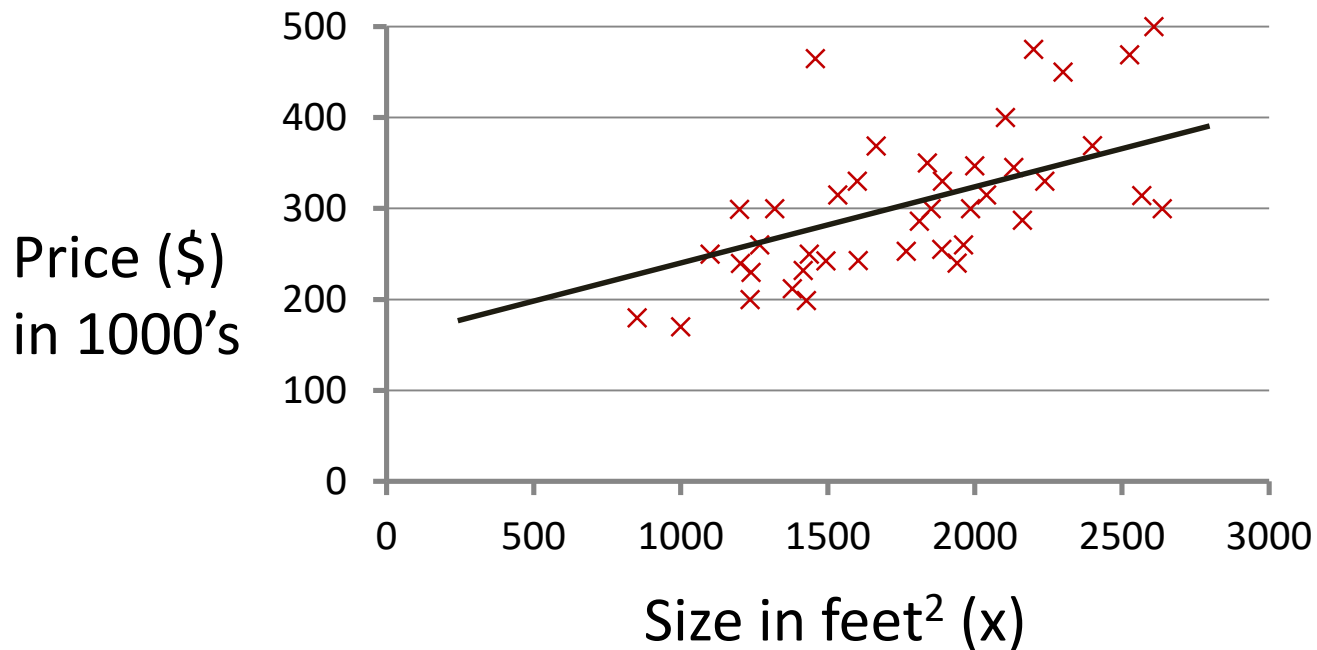
Goal: $\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$



Cost Function for two parameters

$$h_{\theta}(x)$$

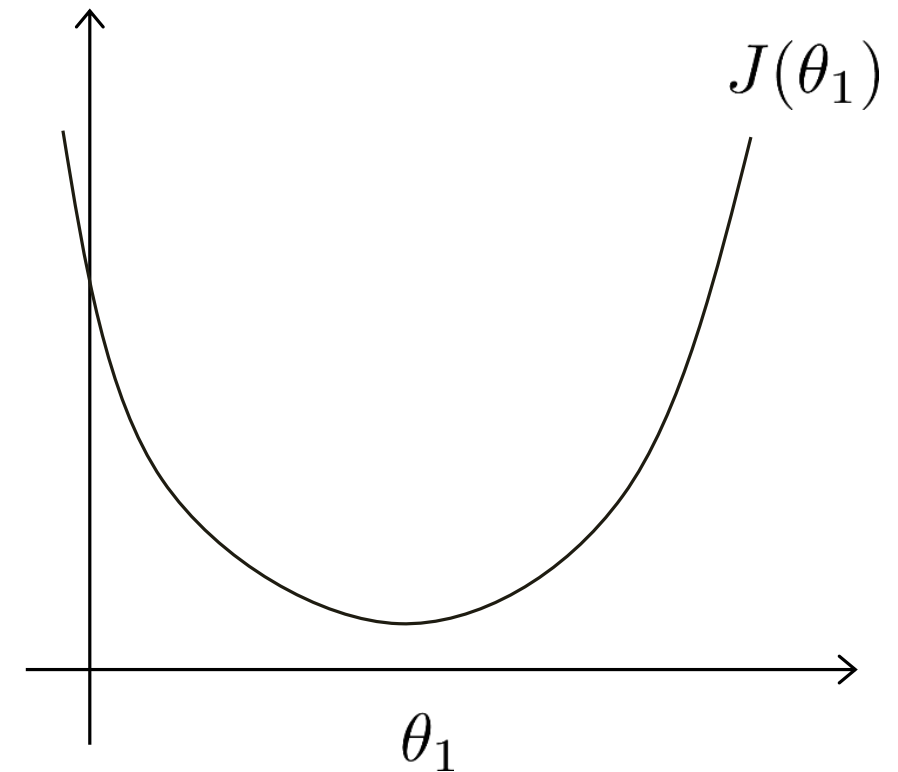
(for fixed θ_0, θ_1 this is a function of x)



$$h_{\theta}(x) = 50 + 0.06x$$

$$J(\theta_0, \theta_1)$$

(function of the parameters)



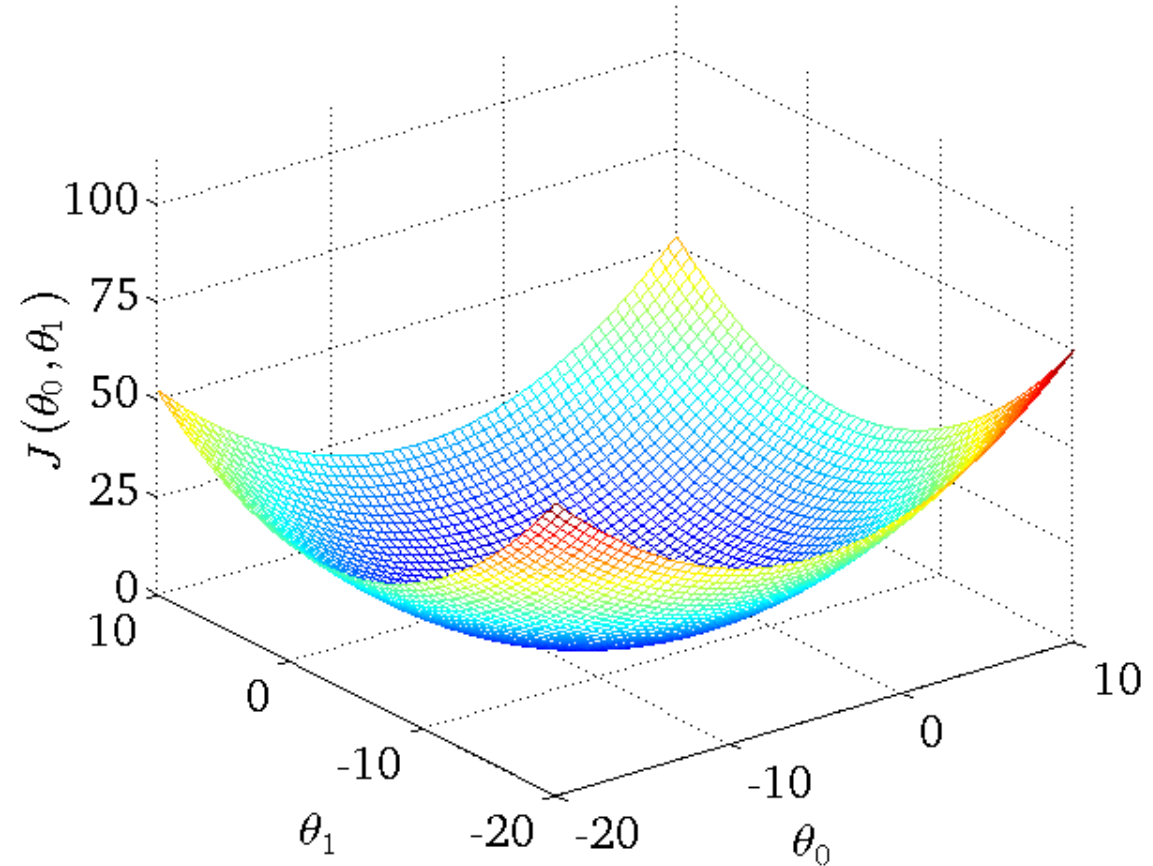
Cost Function for two parameters

- Previously we plotted our cost function by plotting
 - θ_1 vs $J(\theta_1)$
- Now we have two parameters
 - Plot becomes a bit more complicated
 - Generates a 3D surface plot where axis are
 - $X = \theta_1$
 - $Z = \theta_0$
 - $Y = J(\theta_0, \theta_1)$



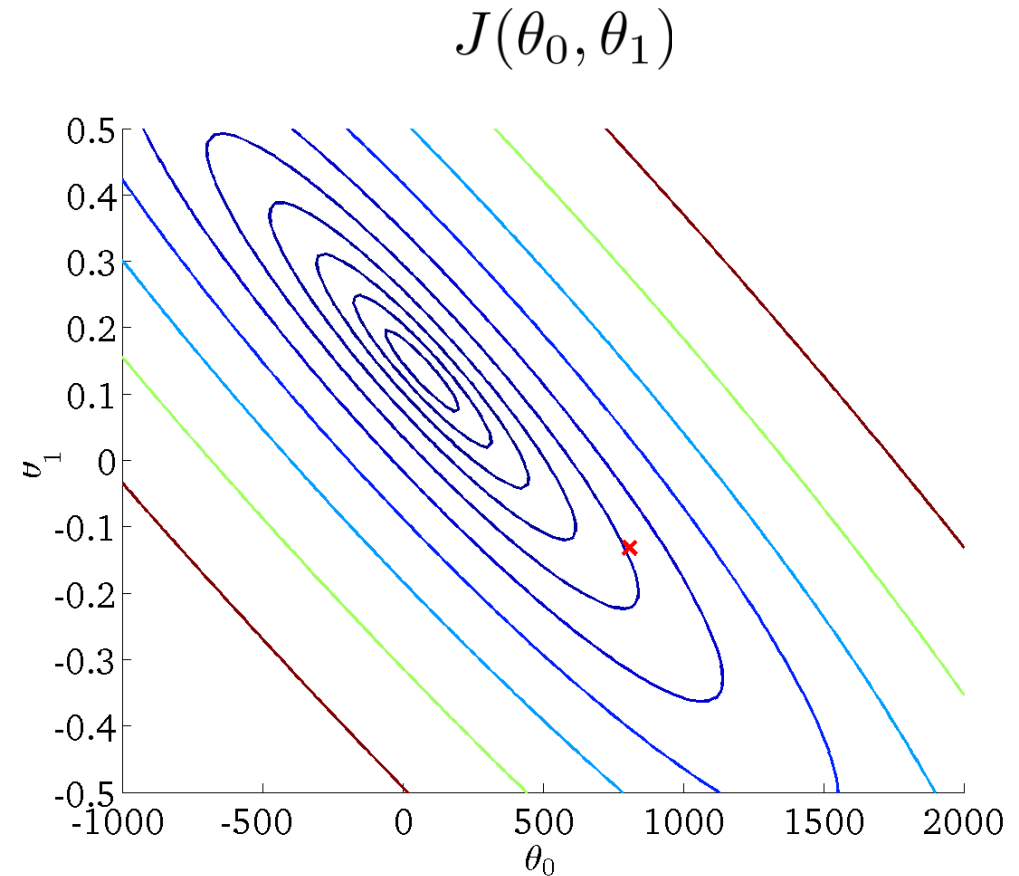
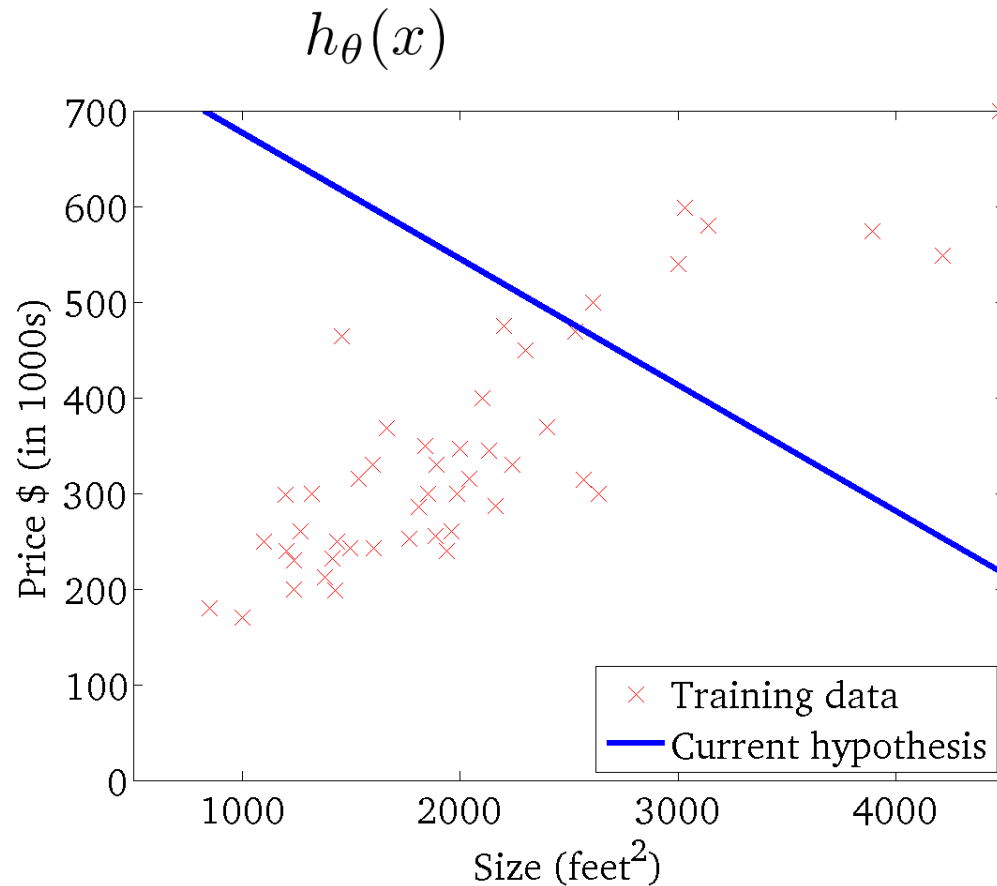
Cost Function for two parameters

- We can see that the height (y) of the graph indicates the value of the cost function,
- we need to find where y is at a minimum



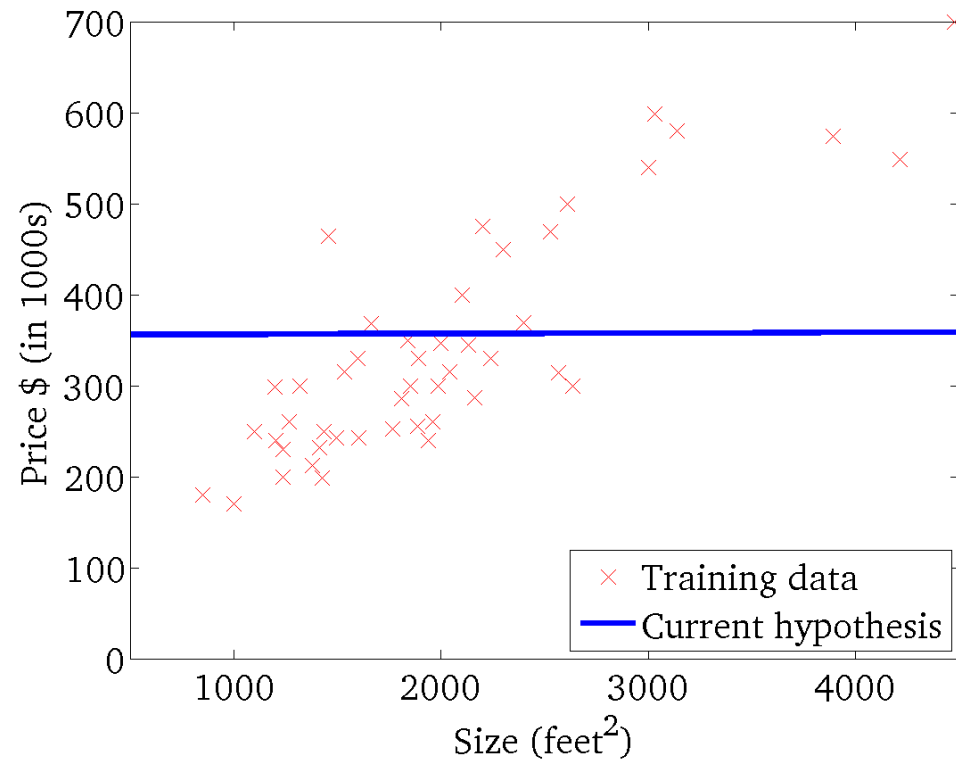
Cost Function for two parameters

- A contour plot is a graphical technique for representing a 3-dimensional surface by plotting constant z slices, called contours, on a 2-dimensional format

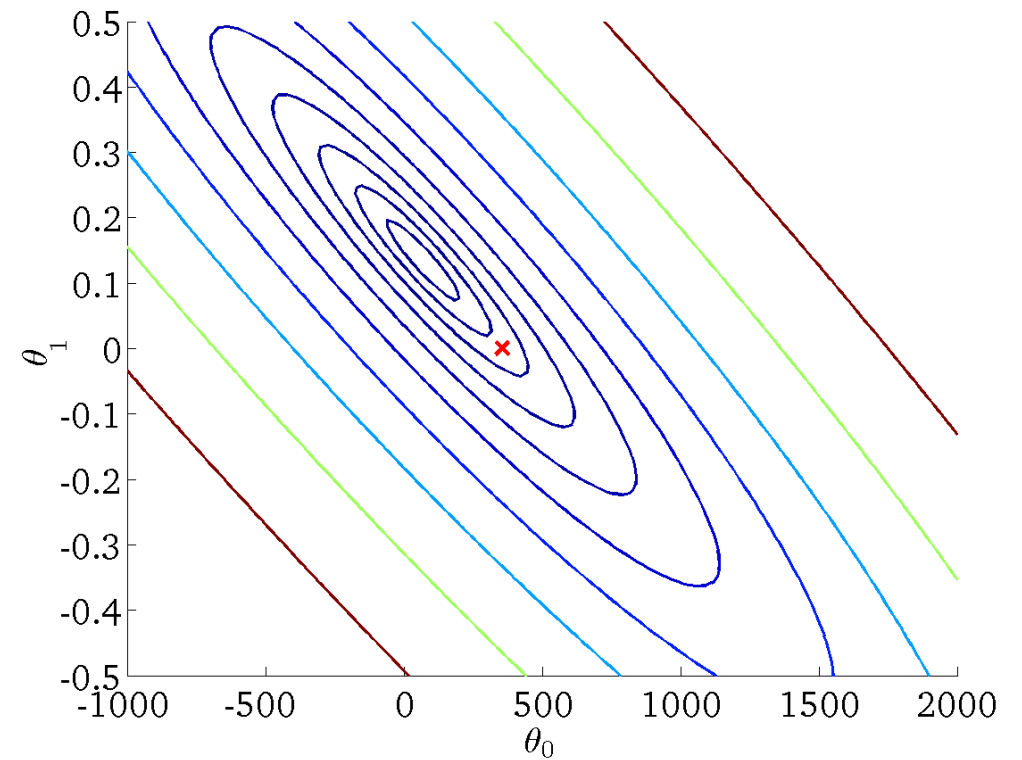


Cost Function for two parameters

$$h_{\theta}(x)$$

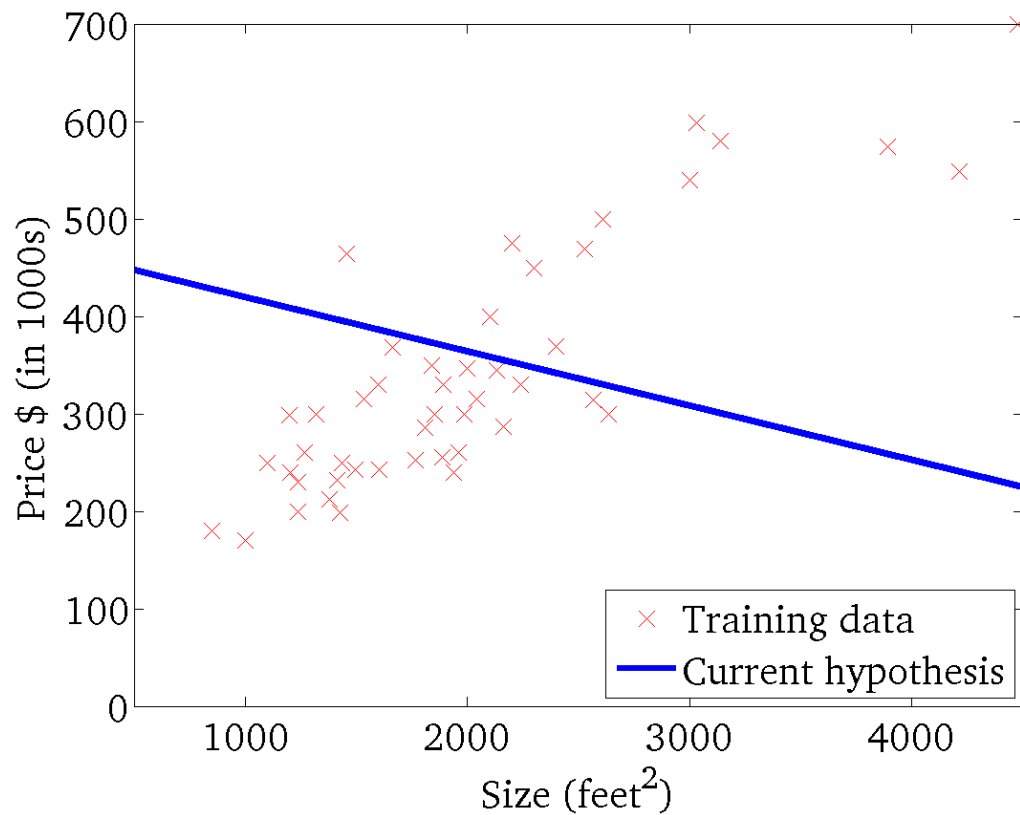


$$J(\theta_0, \theta_1)$$

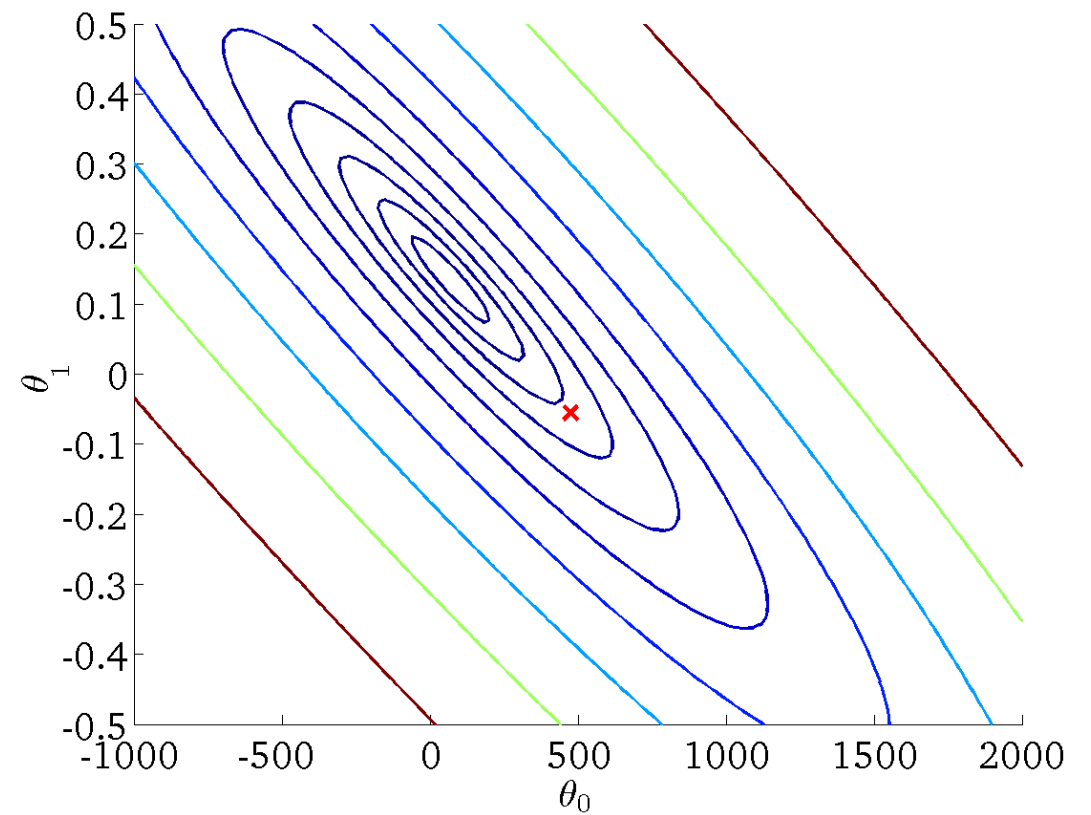


Cost Function for two parameters

$$h_{\theta}(x)$$

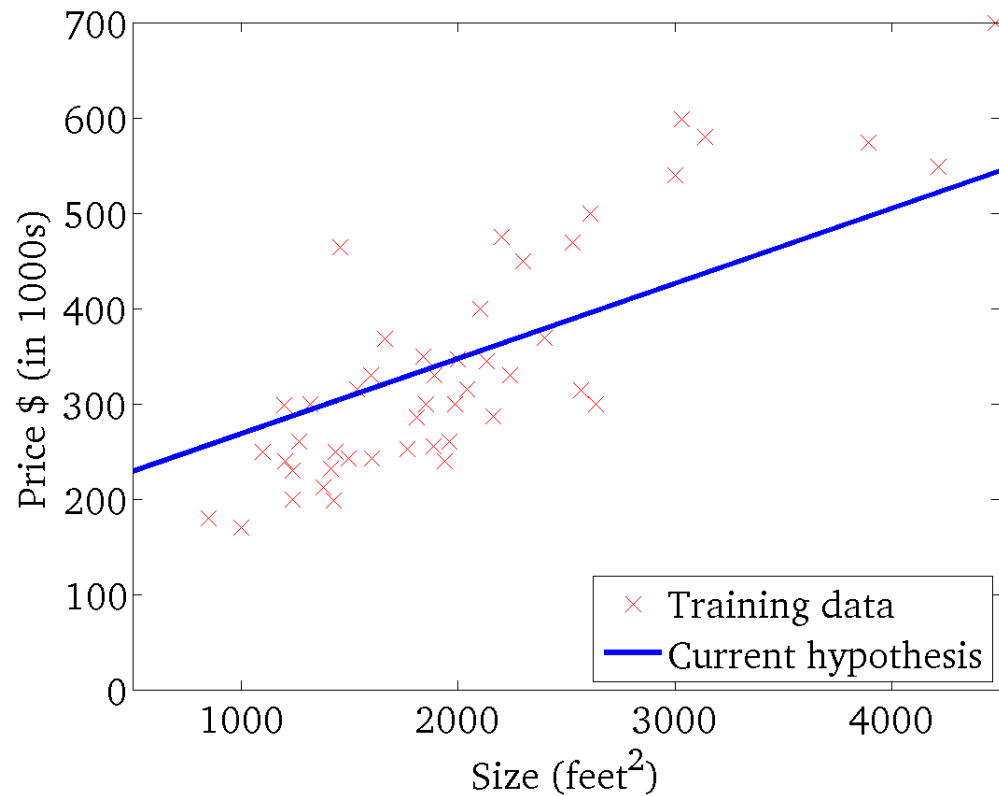


$$J(\theta_0, \theta_1)$$

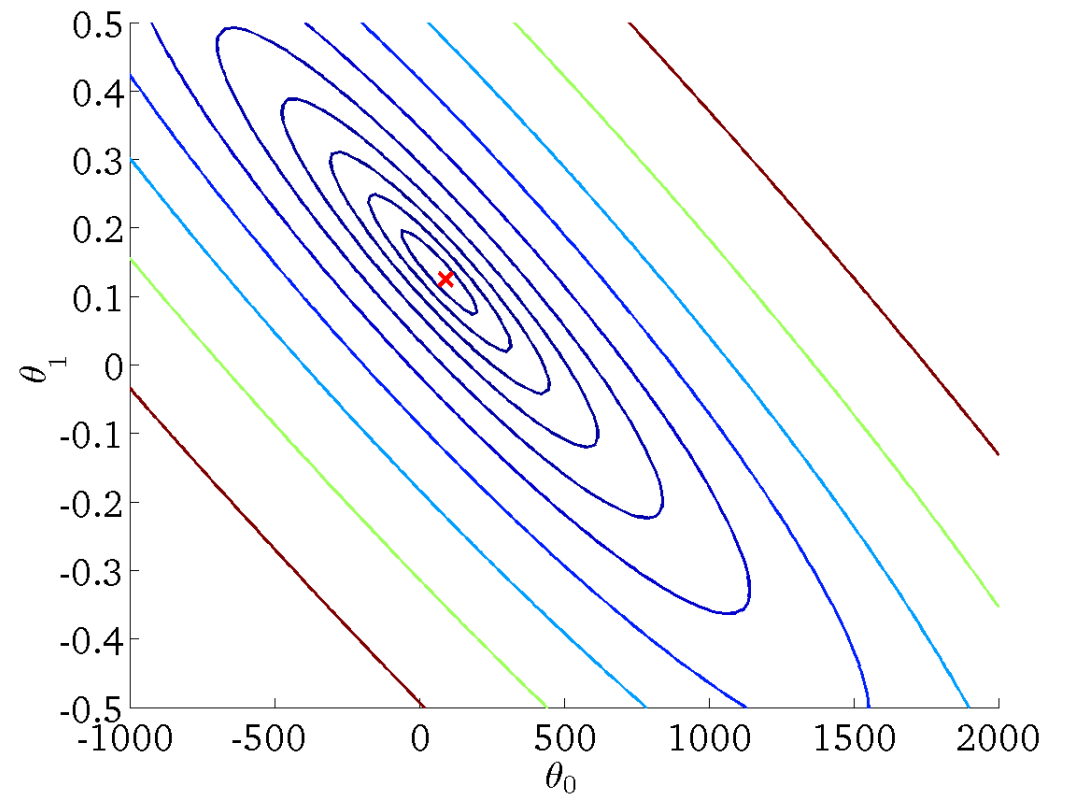


Cost Function for two parameters

$$h_{\theta}(x)$$

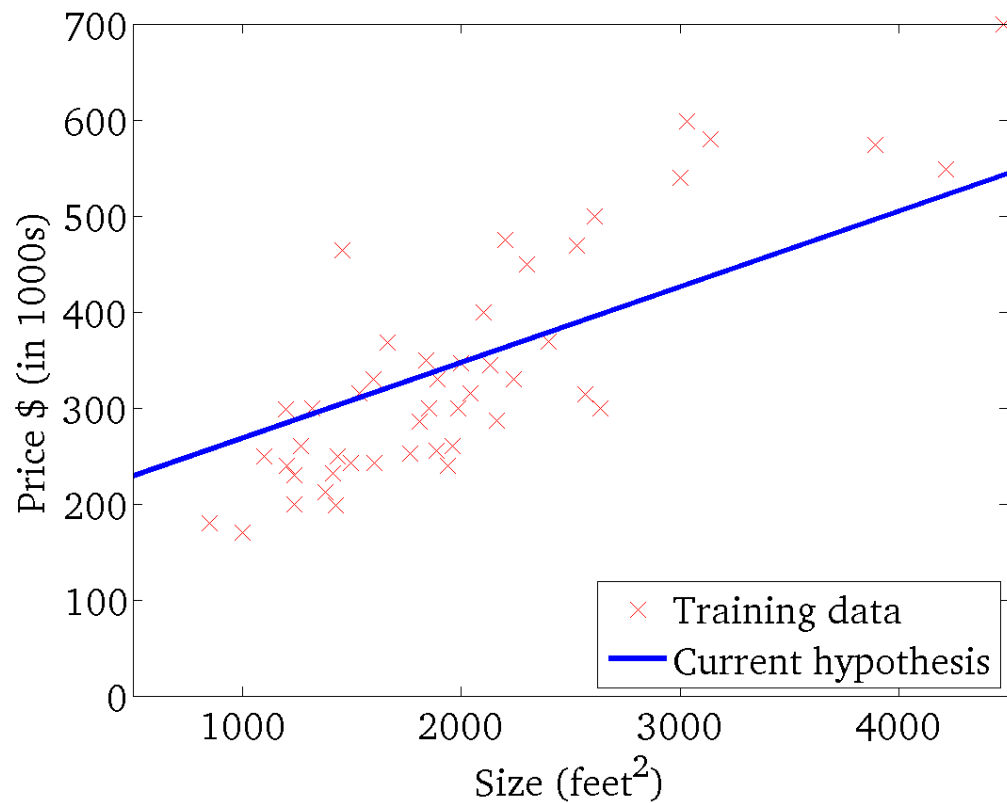


$$J(\theta_0, \theta_1)$$

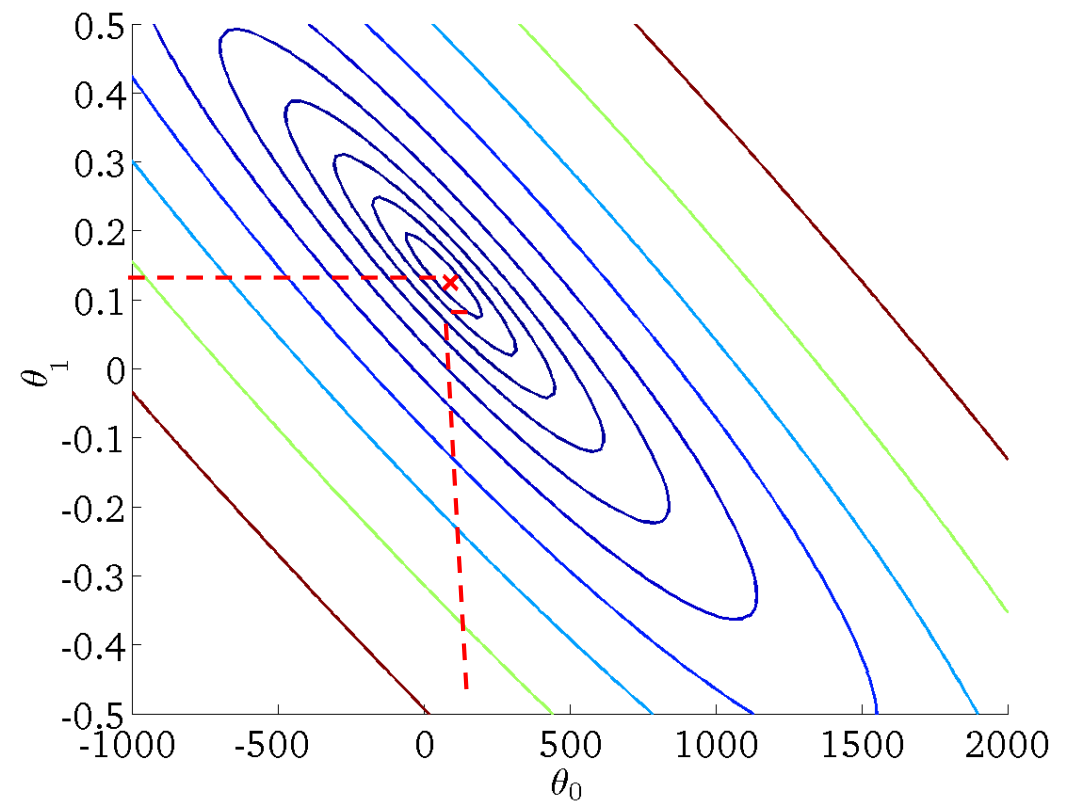


Cost Function for two parameters

$$h_{\theta}(x)$$



$$J(\theta_0, \theta_1)$$



Gradient descent

- We want to get **$\min J(\theta_0, \theta_1)$**
- Gradient descent
 - Used all over machine learning for minimization
- **Outline:**
 - Start with some θ_0, θ_1
 - Keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$ until we hopefully end up at a minimum

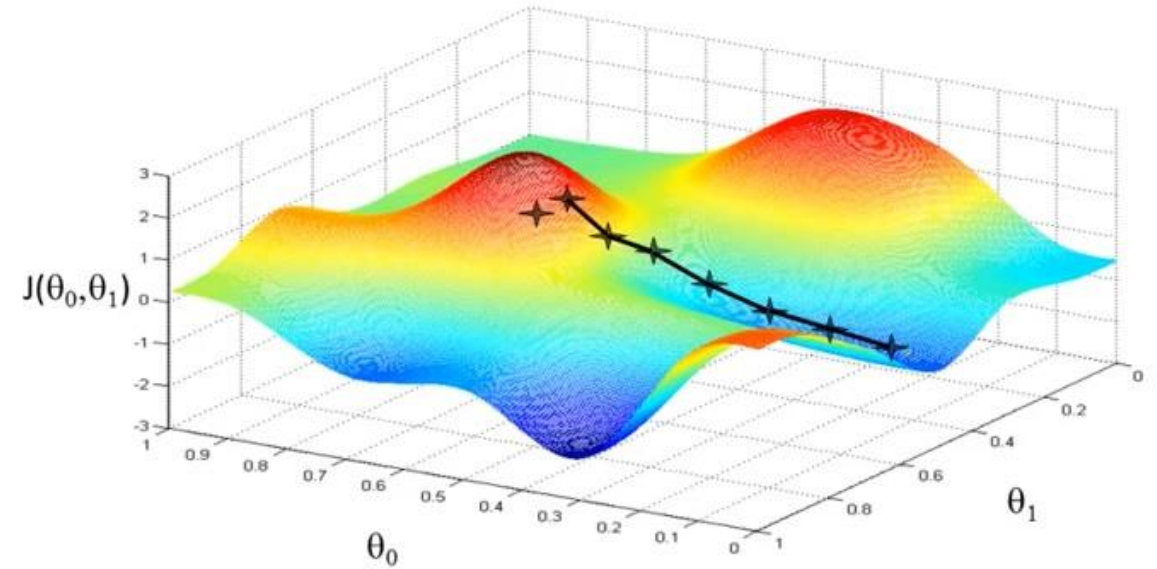
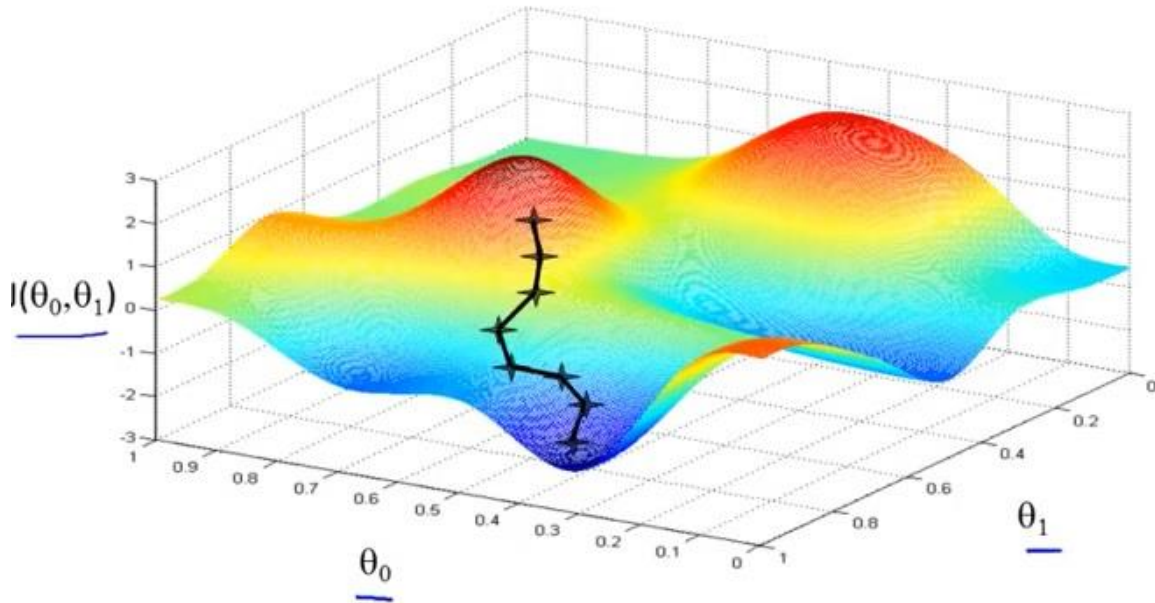


Gradient descent

- Start with **initial guesses**
 - Start at 0,0 (or any other value)
 - Keeping changing θ_0 and θ_1 **a little bit** to try and reduce $J(\theta_0, \theta_1)$
- Each time you change the parameters, you select the gradient which reduces $J(\theta_0, \theta_1)$ the most possible
- Repeat
- Do so until you converge to a local minimum
- Has an interesting property
 - Where you start can determine which minimum you end up
 - Here we can see one initialization point led to one local minimum
 - The other led to a different one



Gradient descent



- One initialization point led to one local minimum.
The other led to a different one

Gradient Descent Algorithm

- Gradient descent is used to minimize the MSE by calculating the **gradient of the cost function**

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ (for $j = 0$ and $j = 1$)
}

Correct: **Simultaneous update**

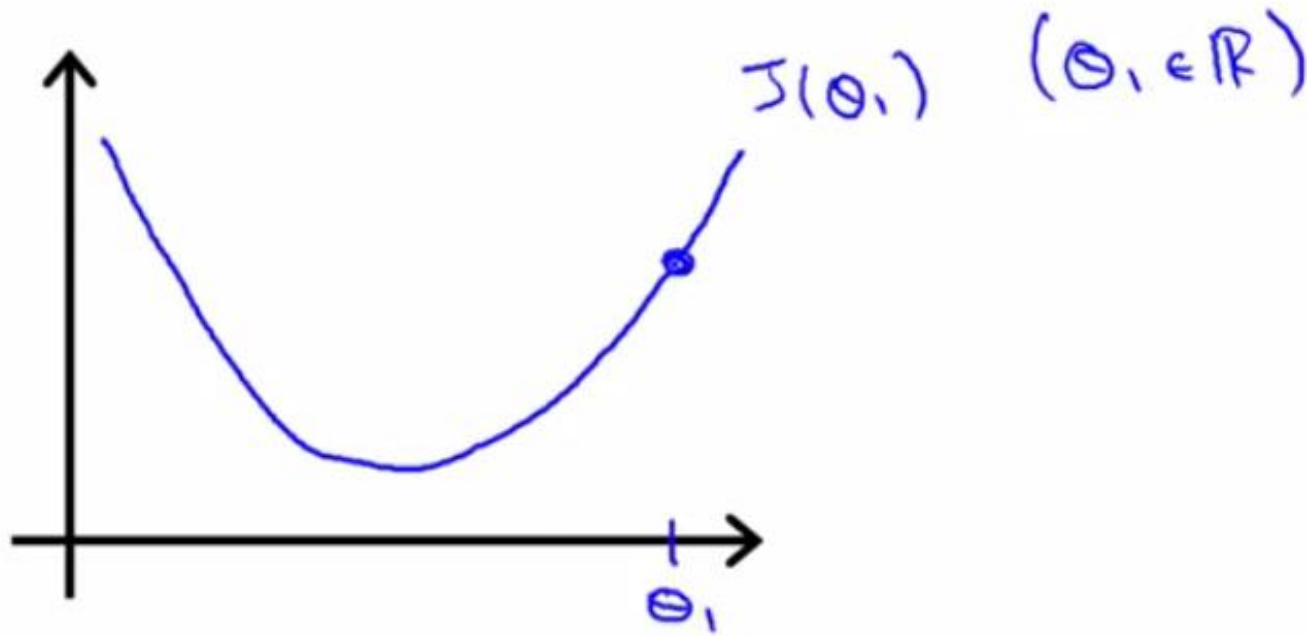
$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 $\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 $\theta_0 := \text{temp0}$
 $\theta_1 := \text{temp1}$

Incorrect:

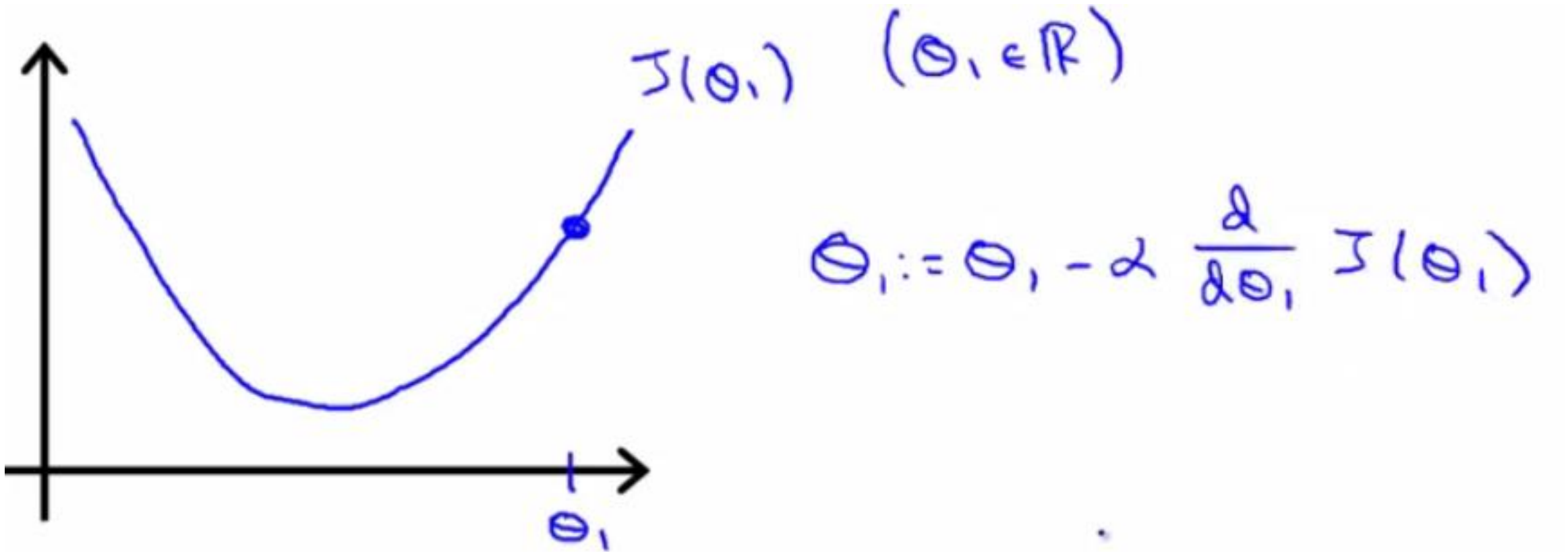
$\text{temp0} := \theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$
 $\theta_0 := \text{temp0}$
 $\text{temp1} := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$
 $\theta_1 := \text{temp1}$



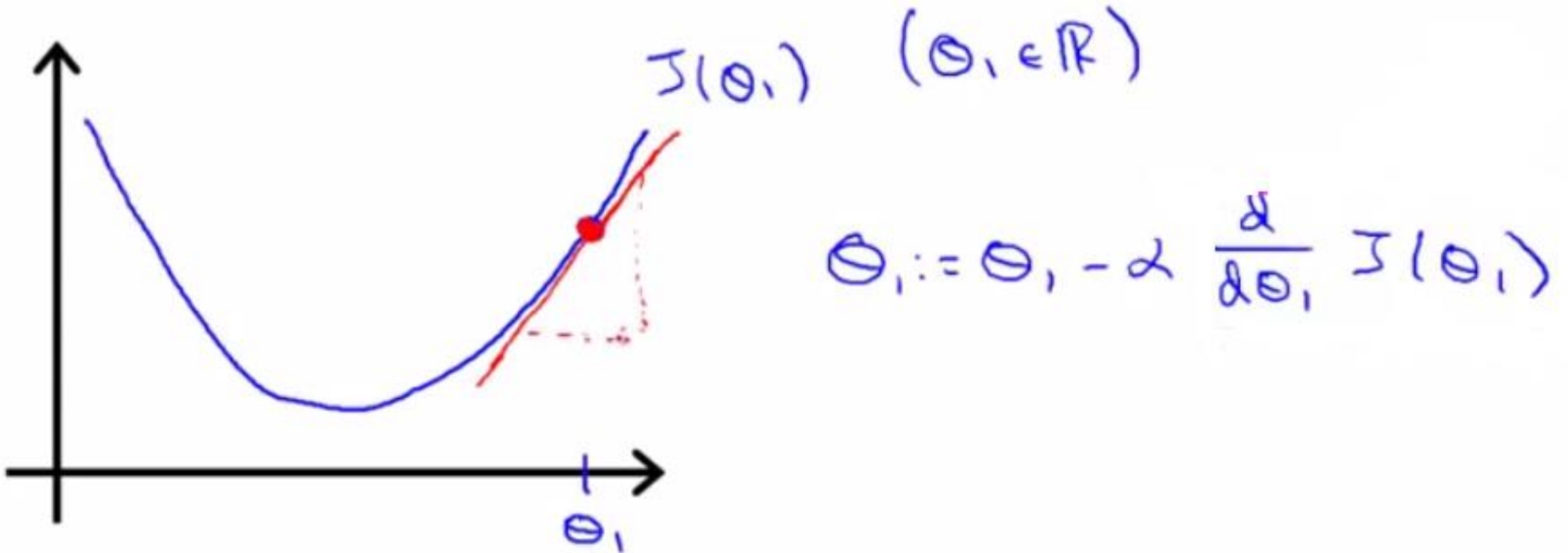
Gradient Descent Algorithm



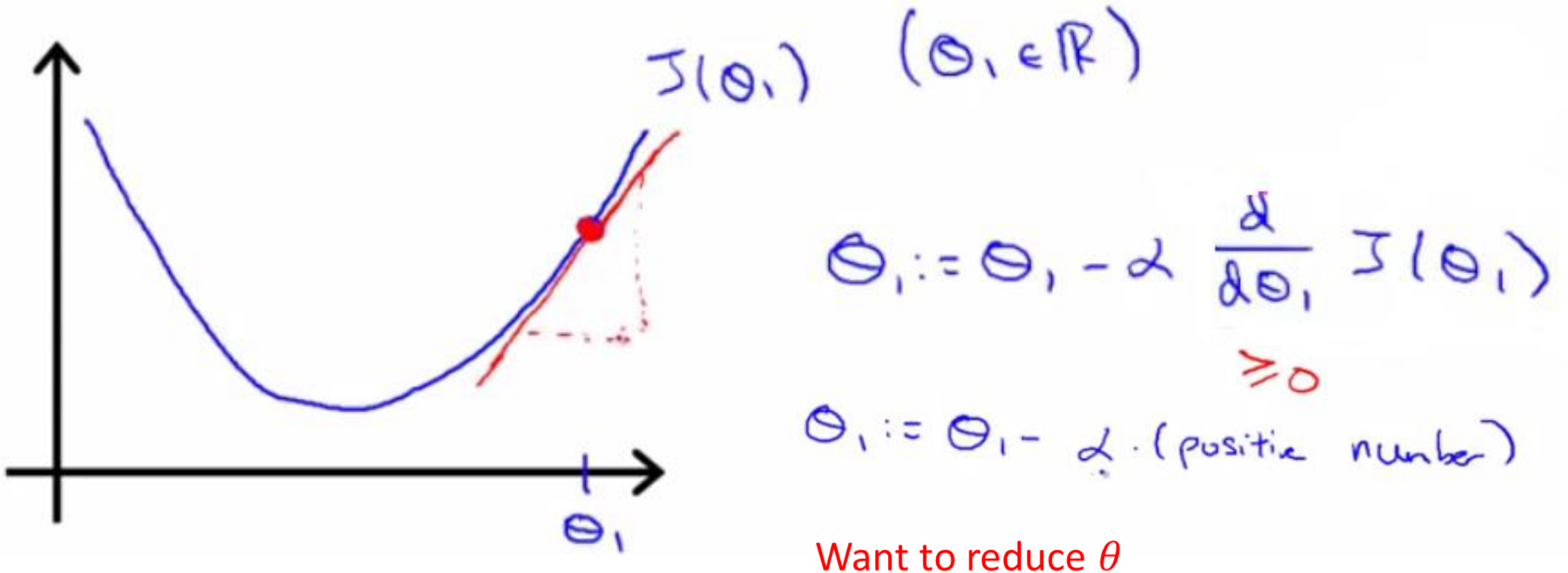
Gradient Descent Algorithm



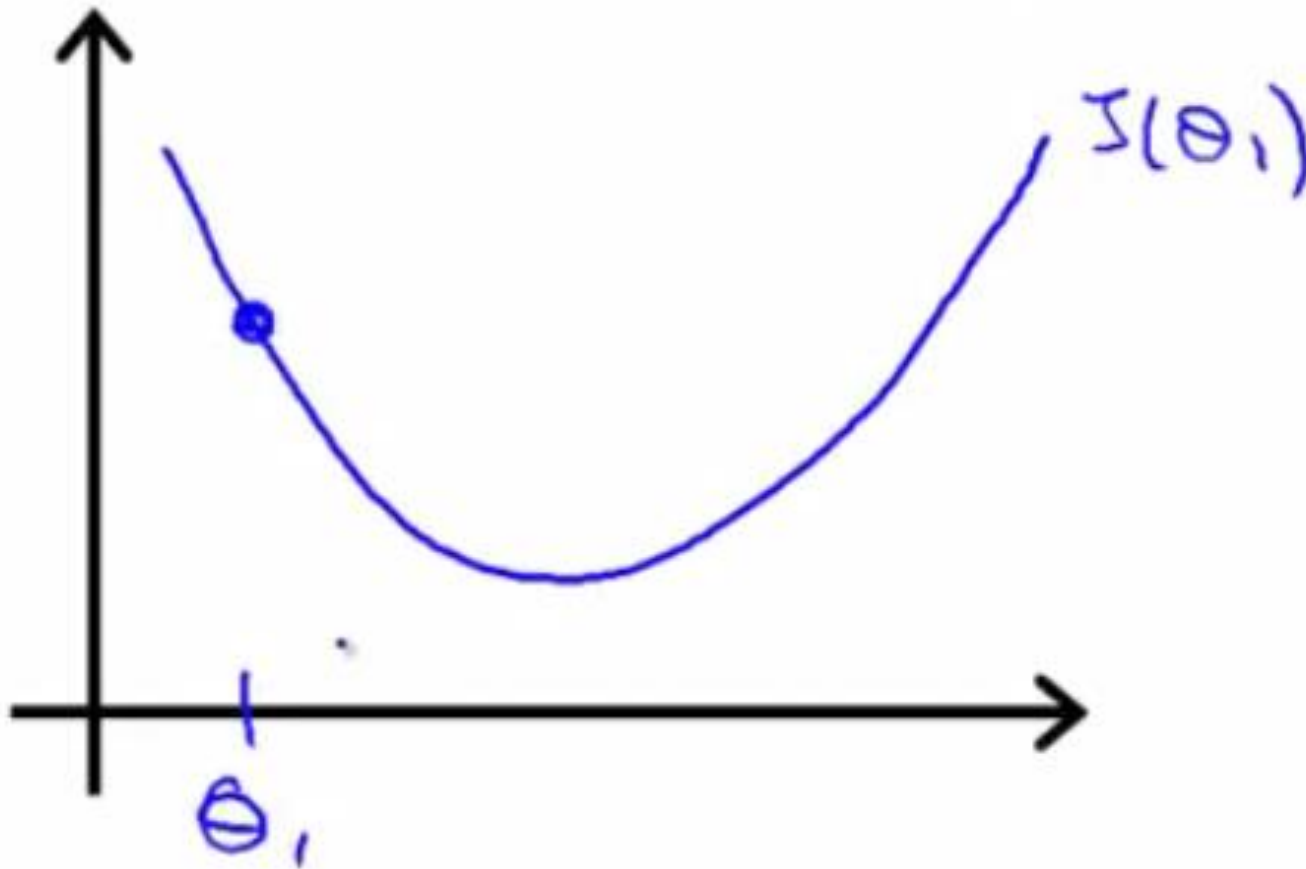
Gradient Descent Algorithm



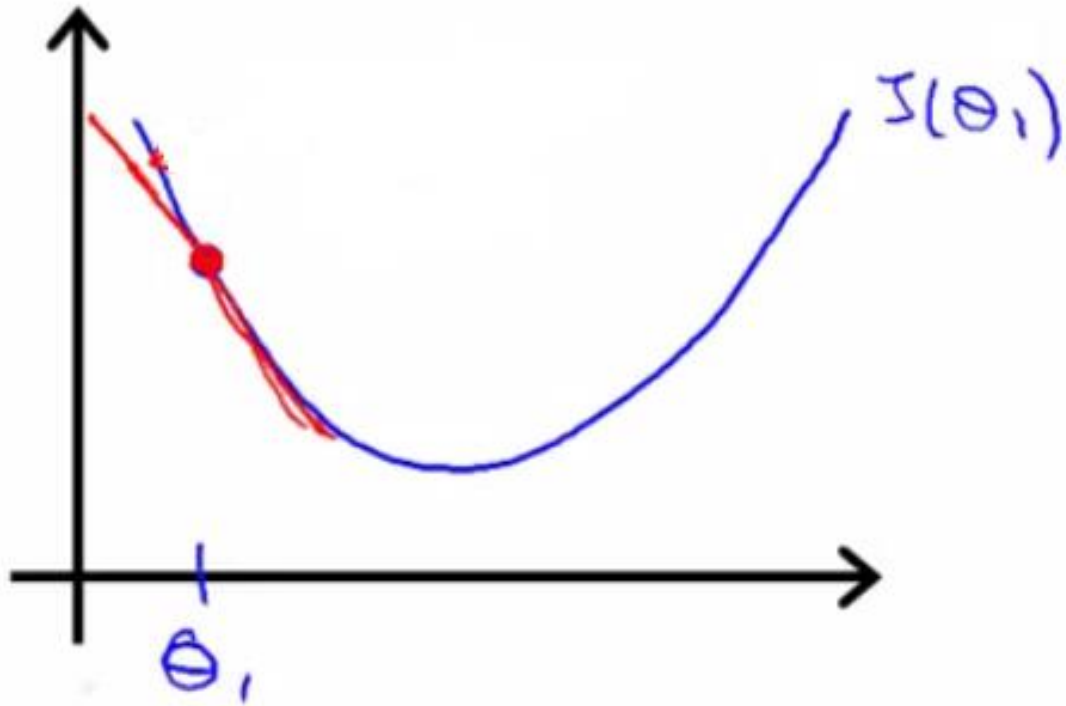
Gradient Descent Algorithm



Gradient Descent Algorithm



Gradient Descent Algorithm



$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

$$\frac{d}{d\theta_1} J(\theta_1) \leq 0$$

$$\theta_1 := \theta_1 - \alpha (\text{negative number})$$

Want to increase θ

Learning Rate

- Here, α is the learning rate, a hyperparameter
- It controls how big steps we made
- If α is small, we will take tiny steps
- If α is big, we have an aggressive gradient descent



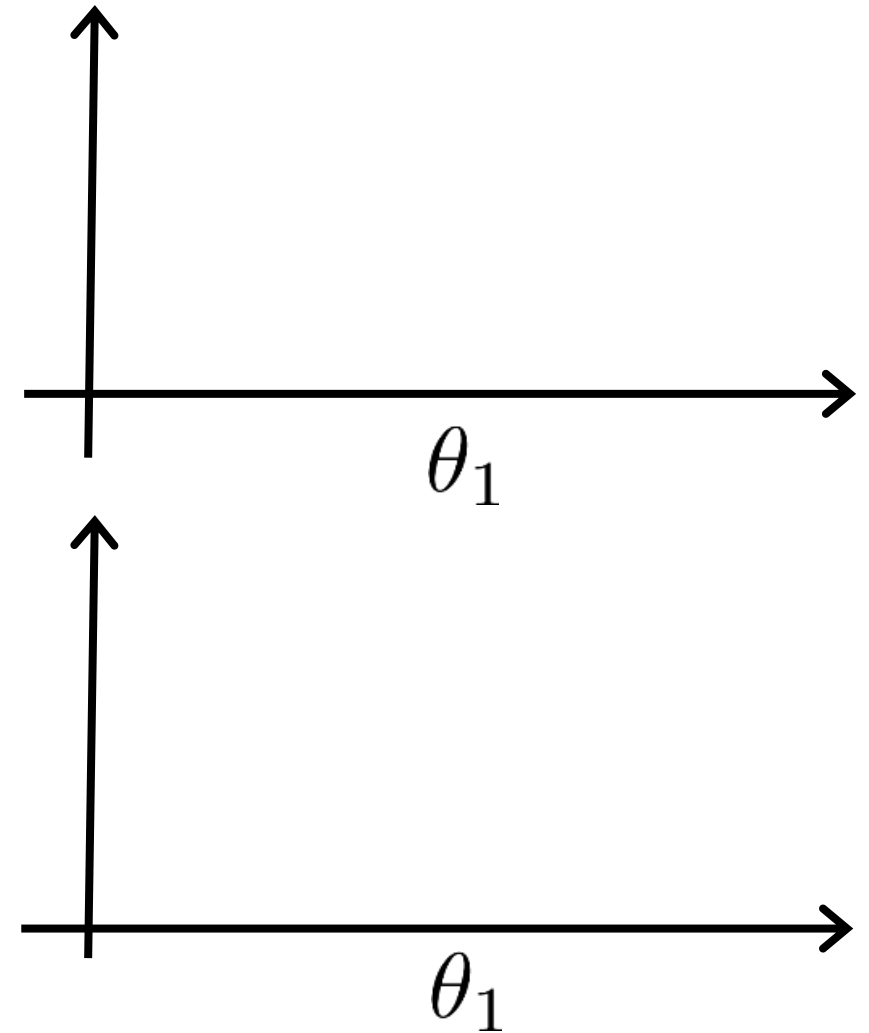
Learning Rate

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

Higher training time

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

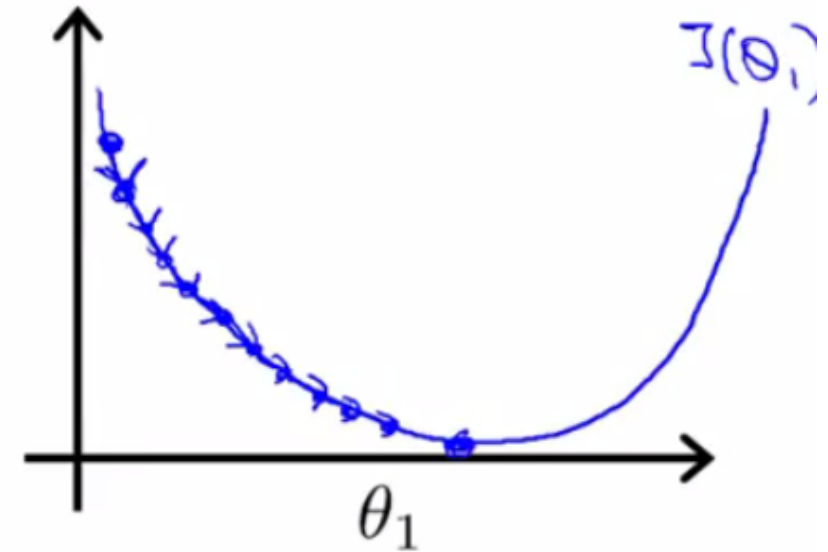


Learning Rate

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

Higher training time



If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

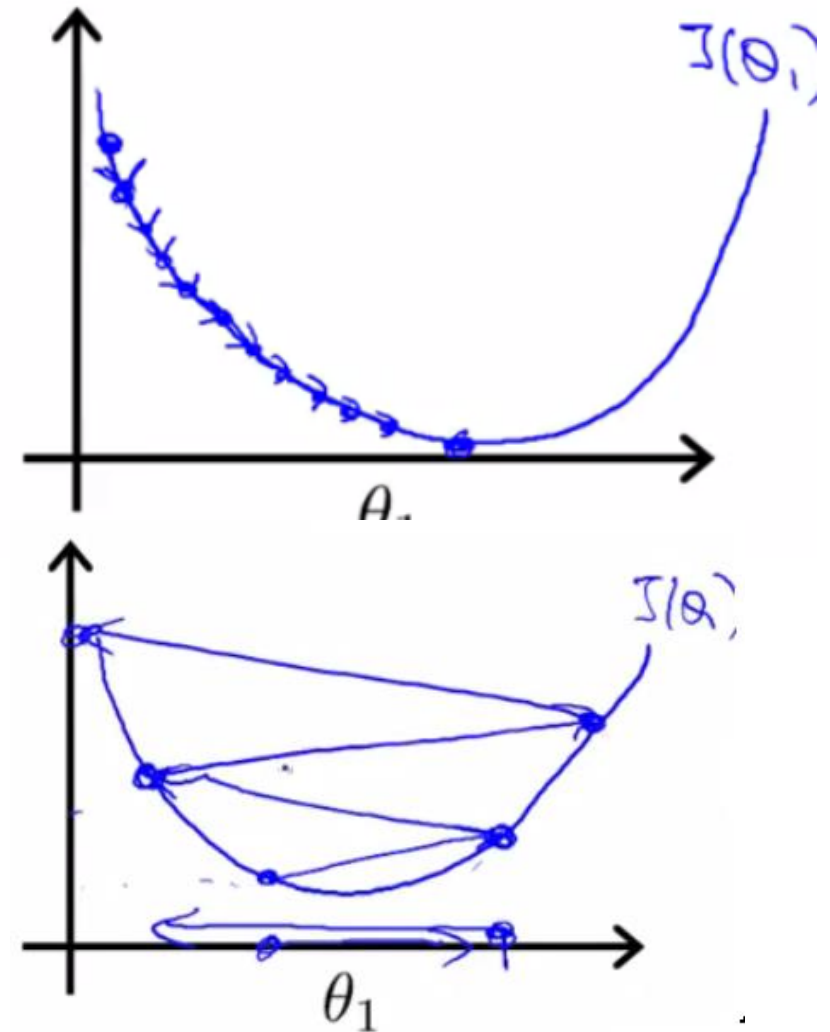
Learning Rate

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

Higher training time

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

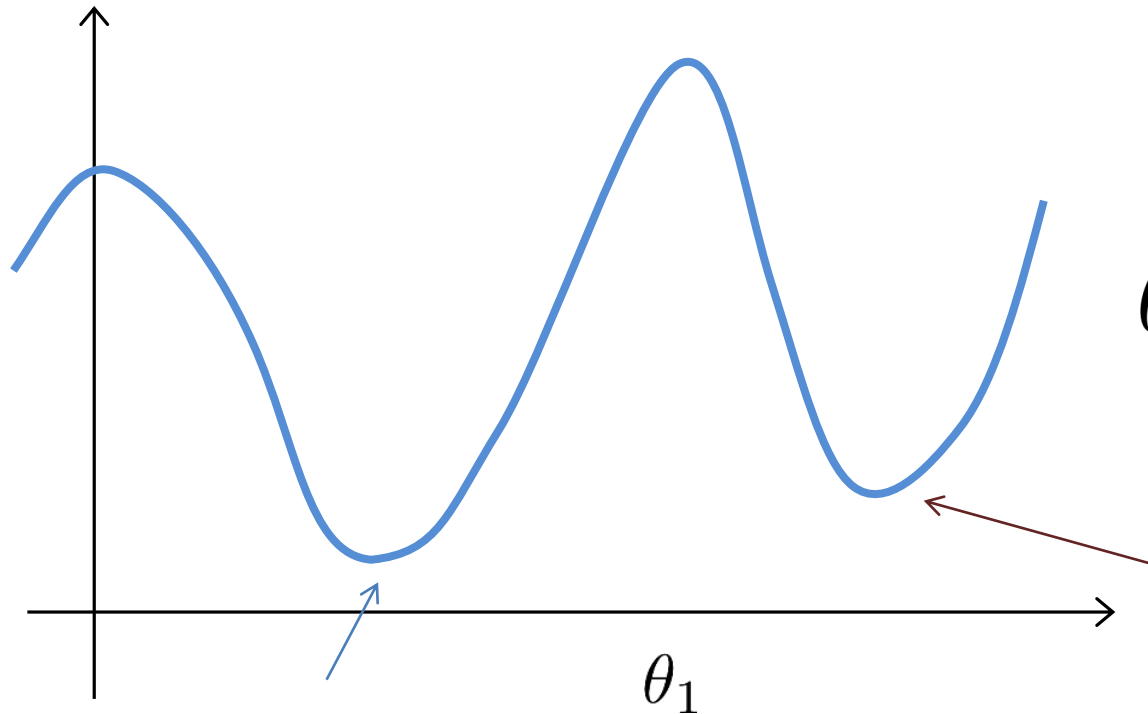


Local Minima

- Local minimum: value of the loss function is minimum at that point in a local region.
- Global minima: value of the loss function is minimum *globally* across the entire domain the loss function



Local Minima



$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

at local minima

$$\theta_1 := \theta_1 - \alpha \cdot 0$$

$$\theta_1 := \theta_1$$

Gradient Descent Calculation

repeat until convergence {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \quad (\text{for } j = 0 \text{ and } j = 1)$$

}

$$\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) = \frac{\partial}{\partial \theta_j} \left[\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \right]$$

$$= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2$$

$$\theta_0, j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1, j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$



Gradient Descent Calculation

Gradient descent algorithm

repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

}



- *Reference:*

- Andrew NG Lectures on Machine Learning, Stanford University

